CS 405: Computer Architecture

Taught by Prof. Ashutosh Srivastava

References:
- Computer System Architecture by M. Morris Mano
- Computer Architecture and Organization by IIT Kharagpur

Prerequisites:
- Digital Logic Fundamentals
- Assembly Language
- Basics of Microprocessors

Introduction

- Computers have become part and parcel of our daily lives.
  - They are everywhere (embedded systems?)
  - Laptops, tablets, mobile phones, intelligent appliances.
- It is required to understand how a computer works.
  - What are there inside a computer?
  - How does it work?
- We distinguish between two terms: *Computer Architecture* and *Computer Organization*.

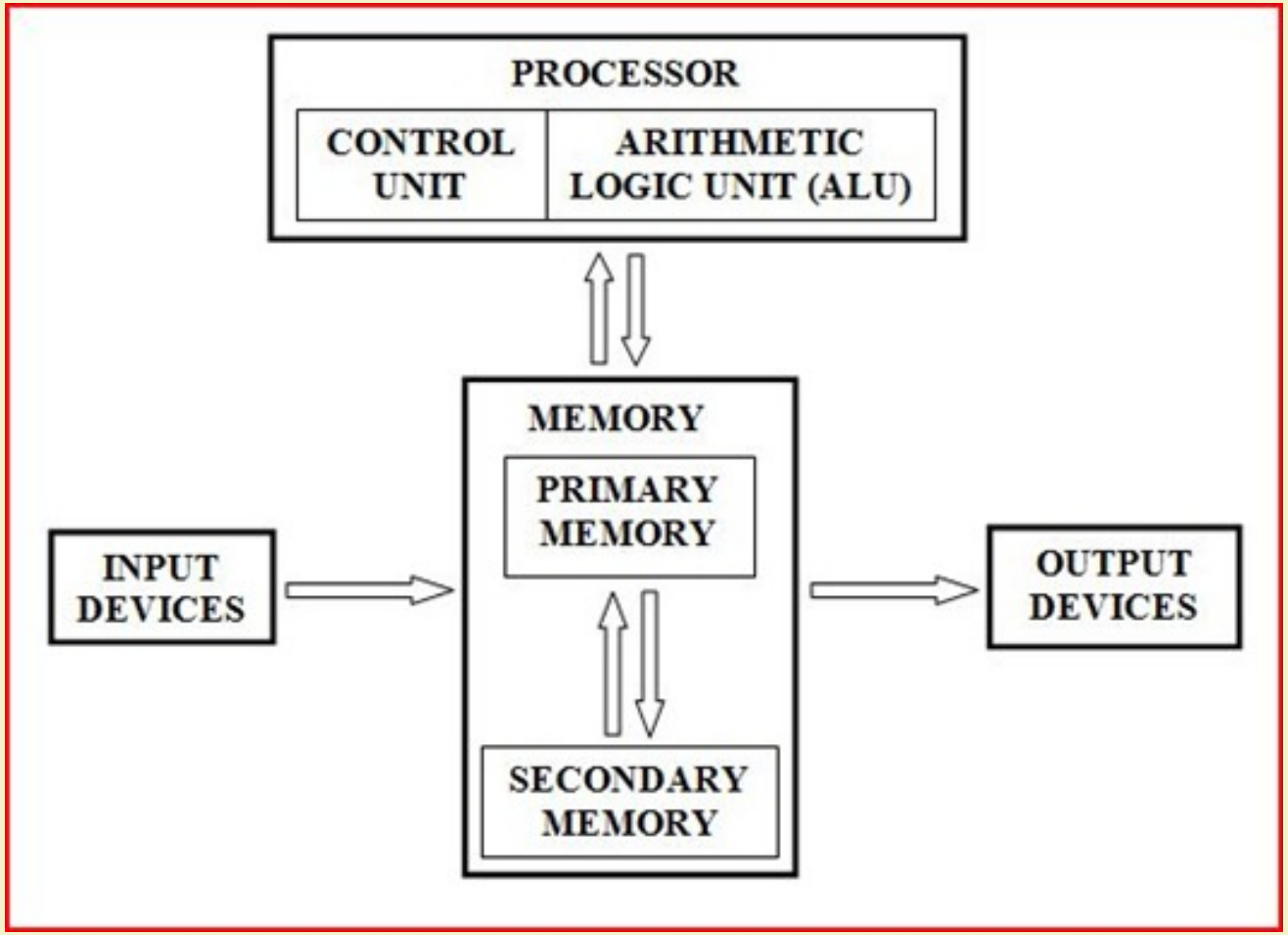
- Computer Organization:
  - Design of the components and functional blocks using which computer systems are built.
  - *Analogy*: civil engineer's task during building construction (cement, bricks, iron rods, and other building materials).
- Computer Architecture:
  - How to integrate the components to build a computer system to achieve a desired level of performance.
  - *Analogy*: architect's task during the planning of a building (overall layout, floorplan, etc.).

Historical Perspective

- Constant quest of building automatic computing machines have driven the development of computers.
  - ***Initial efforts***: mechanical devices like pulleys, levers and gears.
  - ***During World War II***: mechanical relays to carry out computations.
  - ***Vacuum tubes developed***: first electronic computer called ENIAC.
  - ***Semiconductor transistors developed*** and journey of miniaturization began.
    - SSI → MSI → LSI → VLSI → ULSI → …. Billions of transistors per chip

Simplified Block Diagram of a Computer System

- All instructions and data are stored in memory.
- An instruction and the required data are brought into the processor for execution.
- Input and Output devices interface with the outside world.
- Referred to as *von-Neumann architecture*.



- **Inside the Processor**
  - Also called *Central Processing Unit* (CPU).
  - Consists of a *Control Unit* and an *Arithmetic Logic Unit* (ALU).
    - All calculations happen inside the ALU.
    - The Control Unit generates sequence of control signals to carry out all operations.
  - The processor fetches an instruction from memory for execution.
    - An instruction specifies the exact operation to be carried out.
    - It also specifies the data that are to be operated on.
    - A program refers to a set of instructions that are required to carry out some specific task (e.g. sorting a set of numbers).

- What is the role of ALU?
  - It contains several registers, some general-purpose and some special-purpose, for temporary storage of data.
  - It contains circuitry to carry out logic operations, like AND, OR, NOT, shift, compare, etc.
  - It contains circuitry to carry out arithmetic operations like addition, subtraction, multiplication, division, etc.
  - During instruction execution, the data (operands) are brought in and stored in some registers, the desired operation carried out, and the result stored back in some register or memory.

- What is the role of control unit?
  - Acts as the nerve center that senses the states of various functional units and sends control signals to control their states.
  - To carry out a specific operation (say, R1 ← R2 + R3), the control unit must generate control signals in a specific sequence.
    - Enable the outputs of registers R2 and R3.
    - Select the addition operation.
    - Store the output of the adder circuit into register R1.
  - When an instruction is fetched from memory, the operation (called *opcode*) is decoded by the control unit, and the control signals issued.

- **Inside the Memory Unit**
  - Two main types of memory subsystems.
    - *Primary or Main memory*, which stores the active instructions and data for the program being executed on the processor.
    - *Secondary memory*, which is used as a backup and stores all active and inactive programs and data, typically as *files*.
  - The processor only has direct access to the primary memory.
  - In reality, the memory system is implemented as a hierarchy of several levels.
    - L1 cache, L2 cache, L3 cache, primary memory, secondary memory.
    - Objective is to provide faster memory access at affordable cost.

– Various different types of memory are possible.

a) Random Access Memory (RAM), which is used for the cache and primary memory sub-systems. Read and Write access times are independent of the location being accessed.

b) Read Only Memory (ROM), which is used as part of the primary memory to store some fixed data that cannot be changed.

c) Magnetic Disk, which uses direction of magnetization of tiny magnetic particles on a metallic surface to store data. Access times vary depending on the location being accessed, and is used as secondary memory.

d) Flash Memory, which is replacing magnetic disks as secondary memory devices. They are faster, but smaller in size as compared to disk.

Timeline for Operating Systems Only for Silicon-Based MOS Transistor IC's Used in Commercial Microcomputers, Since Its Inception

A chronological overview of landmark operating systems in the history of the microcomputer.

Year	OS	Why it matters
1974	**CP/M**	Beginning of commercially successful microcomputer OSs
1978	**Apple DOS**	Disk OS for a mass-market microcomputer
1981	**MS-DOS / PC-DOS**	Establishes the IBM-compatible PC OS ecosystem
1984	**Macintosh System Software**	Major GUI microcomputer OS
1985	**Windows 1.0**	Beginning of Microsoft's GUI OS lineage
1991	**Linux**	Major open Unix-like OS for microcomputers and later servers/embedded systems
1993	**Windows NT**	Modern Windows NT architecture begins
2001	**Mac OS X**	Unix-based modern Macintosh OS
2007	**iOS**	Mobile computing OS becomes mainstream
2008	**Android**	Linux-based mobile OS becomes a major commercial platform

8085 Microprocessor - Key Specifications

Data Bus Width	8-bit
Address Bus Width	16-bit
Memory Capacity	64 KB - (0000)H to (FFFF)H
Clock Frequency	3 MHz
Power Supply	5 Volts
Packaging	40-pin DIP (NMOS)
Instruction Set	74 Instructions

8085 Microprocessor - Register Set

Accumulator (8)	Flags
B (8)	C (8)
D (8)	E (8)
H (8)	L (8)
Program Counter (PC) (16)	
Stack Pointer (SP) (16)	

- Six general-purpose register - B, C, D, E, H, L
- They can be combined as register pairs - BC, DE, HL
- Result of an operation is stored in the Accumulator
- Flag Register has 5 flags - Zero (Z), Carry (CY), Sign (S), Parity (P), Auxiliary Flag (AC)
- HL register pair is used to store memory addresses and is referenced as M
- Program Counter stores address of next instruction byte to be fetched
- Stack Pointer points to a location in Stack memory.

8085 Microprocessor - Addressing Modes

1. **Direct Addressing Mode**: The memory address of the data is explicitly provided in the instruction.
   > `LDA 2050H`

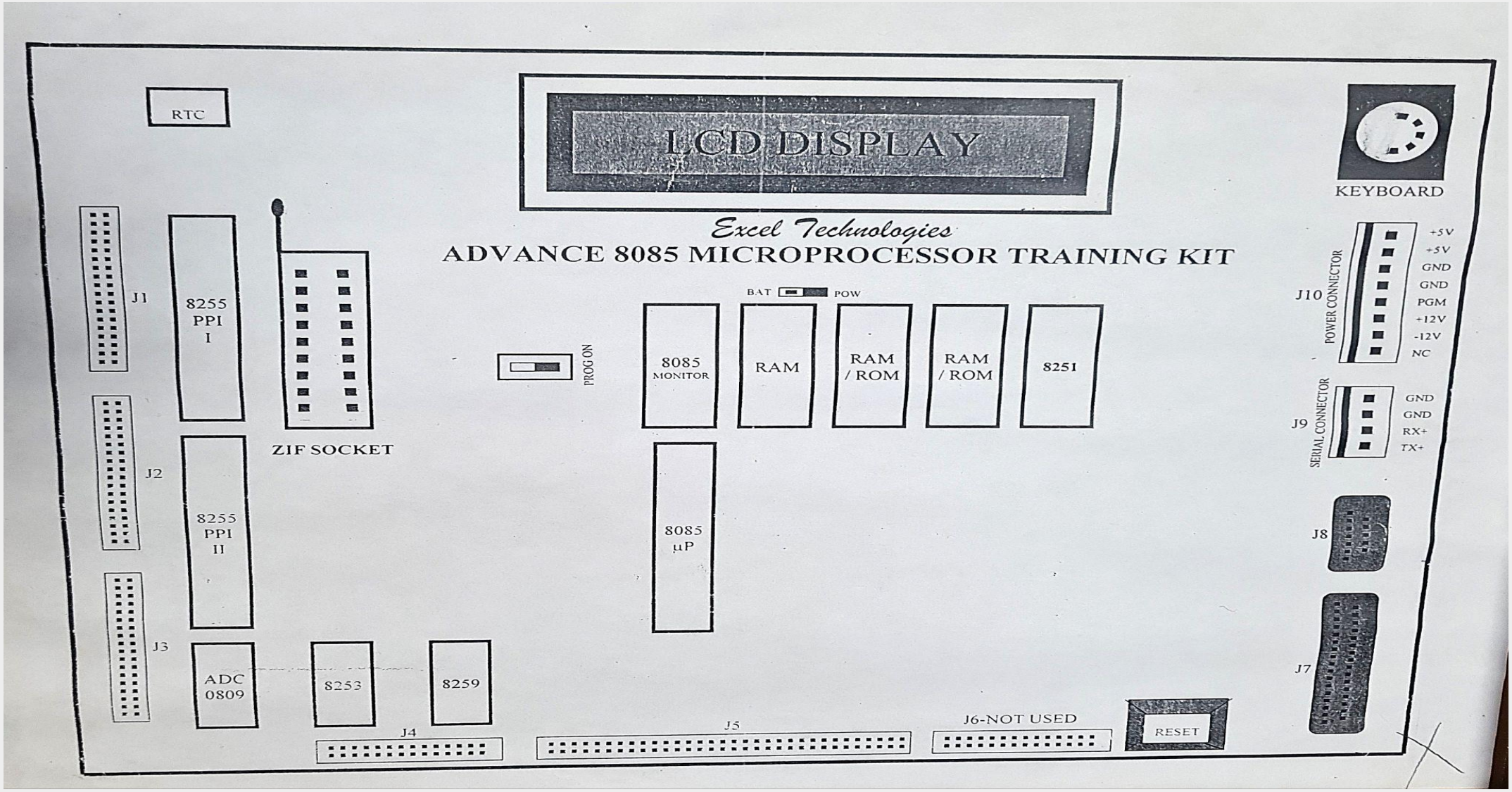
2. **Indirect Addressing Mode**: Memory address of data is stored inside the HL register pair.
   > `MOV A, M`

3. **Immediate Addressing Mode**: Data comes directly from I/O and is supplied within the instruction.
   > `MVI A, 05H`

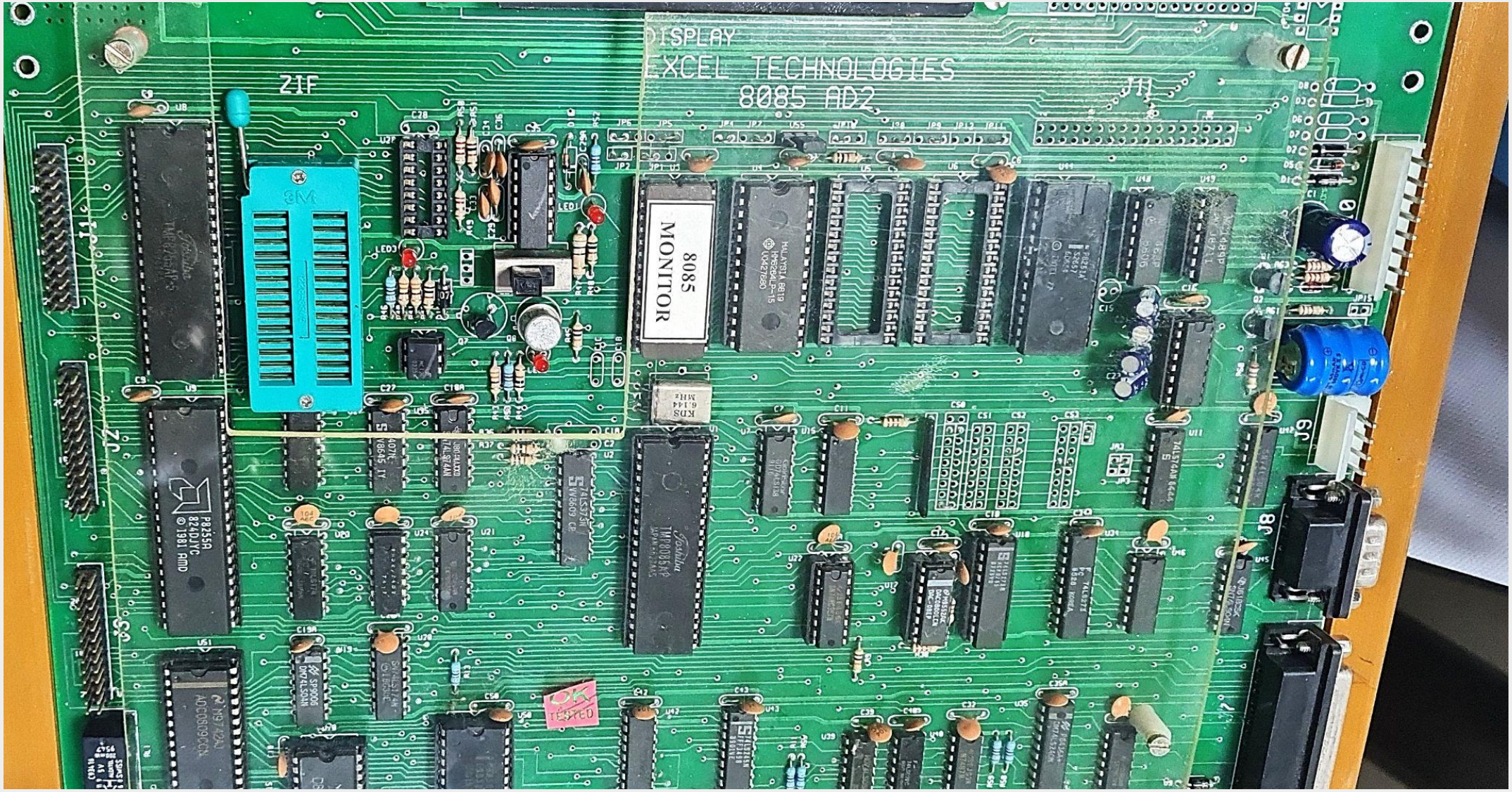
4. **Register Transfer**: Data is moved directly between registers.
   > `MOV A, B`

5. **Implied Addressing Mode**: Operand is implied directly by the opcode (like complement of data, etc.)
   > `CMA`

8085 Microprocessor - Training Kit



8085 Microprocessor - Training Kit



8086 Microprocessor - Key Specifications

Data Bus Width	16-bit
Address Bus Width	20-bit
Memory Capacity	1 MB - (00000)H to (FFFFF)H
Clock Frequency	5 MHz
Power Supply	<= 5 Voltz
Packaging	40-pin DIP (scaled down)
Instruction Set	117 instructions

8086 Microprocessor - BUI and EU

8086 is bifurcated into Bus Interface Unit and Execution Unit. BUI is used to evaluate and identify the effective address (20-bit memory address).

Bus Interface Unit (BUI)

Code Segment (CS)	16 bit
Data Segment (CS)	16 bit
Stack Segment (SS)	16 bit
Extra Segment (ES)	16 bit
Instruction Pointer (IP)	16 bit

Execution Unit (EU)

AX (16)	Higher (8) + Lower (8)
BX (16)	Higher (8) + Lower (8)
CX (16)	Higher (8) + Lower (8)
DX (16)	Higher (8) + Lower (8)
Source Index (SI)	16 bits
Destination Index (DI)	16 bits
Base Pointer (BP)	16 bits
Stack Pointer (SP)	16 bits

8086 Microprocessor - Addressing Modes

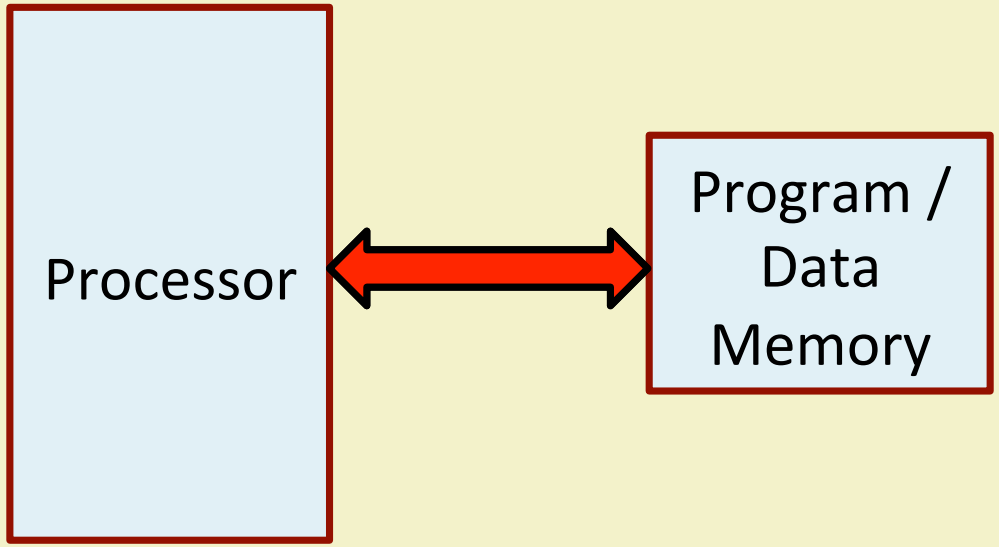
1. **Immediate Addressing Mode**: Data operand is a constant value directly supplied within the instruction.
   > `MOV AX, 0005H`
2. **Register Addressing Mode**: The data operand is stored inside an internal register.
   > `MOV AX, BX`
3. **Direct Addressing Mode**: The 16-bit offset memory address is directly specified inside the instruction.
   > `MOV AX, [2000H]`
4. **Register Indirect Addressing Mode**: The memory address is held inside a base (BX, BP) or index (SI, DI) register.
   > `MOV AX, [BX]`
5. **Based Addressing Mode**: Effective memory address is calculated by adding a displacement to a base register (BX or BP).
   > `MOV AX, [BX + 08H]`
6. **Indexed Addressing Mode**: Effective memory address is calculated by adding a displacement to an index register (SI or DI).
   > `MOV AX, [SI + 04H]`
7. **Based Indexed Addressing Mode**: Effective memory address is calculated by adding the contents of a base register and an index register.
   > `MOV AX, [BX + SI]`
8. **Based Indexed with Displacement**:Effective memory address is calculated by adding a base register, an index register, and a displacement.
   > `MOV AX, [BX + SI + 08H]`
9. **Implied / Implicit Addressing Mode**: Operand is hidden or implicitly defined directly by the opcode itself.
   > `CLC`

Classification of Computer Architecture

- Broadly can be classified into two types:
  a) Von-Neumann architecture
  b) Harvard architecture
- How is a computer different from a calculator?
  - They have similar circuitry inside (e.g. for doing arithmetic).
  - In a calculator, user has to interactively give the sequence of commands.

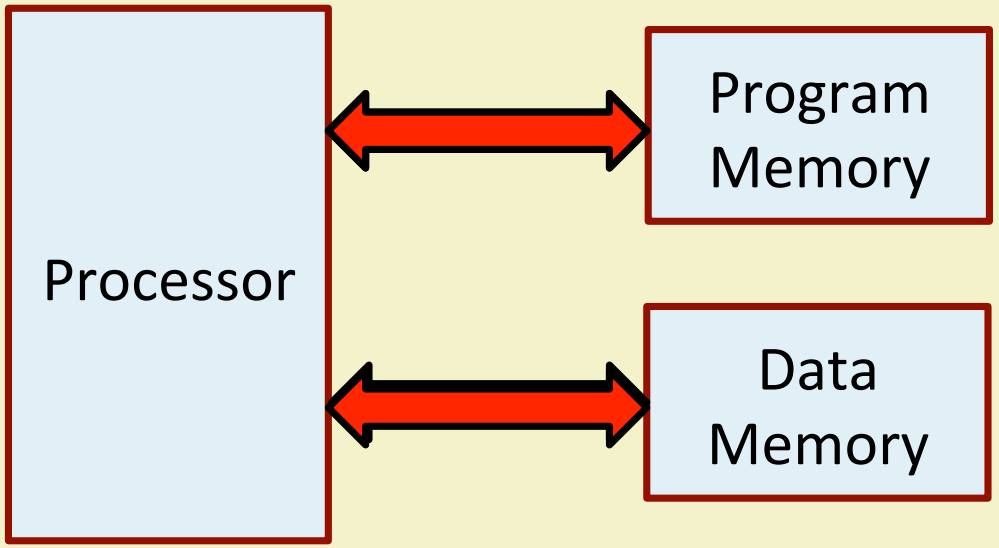
von-Neumann Architecture

- Instructions and data are stored in the same memory module.
- More flexible and easier to implement.
- Suitable for most of the general purpose processors.
- General Disadvantage:
  - The processor-memory bus acts as the bottleneck.
  - All instructions and data are moved back and forth through the pipe.



Harvard Architecture

- Separate memory for program and data.
  - Instructions are stored in program memory and data are stored in data memory.
- Instruction and data accesses can be done in parallel.
- Some microcontrollers and pipelines with separate instruction and data caches follow this concept.
- The processor-memory bottleneck remains.



Emerging Architectures

- Several architectural concepts have been proposed that deviate significantly from the von-Neumann or Harvard model.
- There is a proposal for in-memory computing architecture, where storage and computation can be done in the same functional unit.
  - Memristors are projected to make this possible in the near future.
  - Memristors can be used in high capacity non-volatile resistive memory systems.
  - Memristors within the memory can also be controlled to carry out some computations.

Broad Classification

- Computer architectures have evolved over the years.
  - Features that were developed for mainframes and supercomputers in the 1960s and 1970s have started to appear on a regular basis on later generation microprocessors.
- Two broad classifications of ISA:
  a) Complex Instruction Set Computer (CISC)
  b) Reduced Instruction Set Computer (RISC)

CISC versus RISC Architectures

- Complex Instruction Set Computer (CISC)
  - More traditional approach.
  - Main features:
    - Complex instruction set
    - Large number of addressing modes (R-R, R-M, M-M, indexed, indirect, etc.)
    - Special-purpose registers and Flags (sign, zero, carry, overflow, etc.)
    - Variable-length instructions / Complex instruction encoding
    - Ease of mapping high-level language statements to machine instructions
    - Instruction decoding / control unit design more complex
    - Pipeline implementation quite complex

– CISC Examples:
  - IBM 360/370 (1960-70)
  - VAX-11/780 (1970-80)
  - Intel x86 / Pentium (1985-present)

Only CISC instruction set that survived over generations.

- Desktop PC's / Laptops use these.
- The volume of chips manufactured is so high that there is enough motivation to pay the extra design cost.
- Sufficient hardware resources available today to translate from CISC to RISC internally.

- Reduced Instruction Set Computer (RISC)
  - Very widely used among many manufacturers today.
  - Also referred to as *Load-Store Architecture*.
    - Only LOAD and STORE instructions access memory.
    - All other instructions operate on processor registers.
  - Main features:
    - Simple architecture for the sake of efficient pipelining.
    - Simple instruction set with very few addressing modes.
    - Large number of general-purpose registers; very few special-purpose.
    - Instruction length and encoding uniform for easy instruction decoding.
    - Compiler assisted scheduling of pipeline for improved performance.

– RISC Examples:
  - CDC 6600 (1964)
  - MIPS family (1980-90)
  - SPARC
  - ARM microcontroller family

- Almost all the computers today use a RISC based pipeline for efficient implementation.
  - RISC based computers use compilers to translate into RISC instructions.
  - CISC based computers (e.g. x86) use hardware to translate into RISC instructions.

Introduction

- Most processors execute instructions in a synchronous manner using a clock that runs at a constant *clock rate* or *frequency f*.
- Clock cycle time *C* is the reciprocal of the clock rate *f*:

  $C = 1 / f$
- The clock rate *f* depends on two factors:
  a) The implementation technology used.
  b) The CPU organization used.
- A machine instruction typically consists of a number of elementary micro-operations that vary in number and complexity depending on the instruction and the CPU organization used.

- A micro-operation is an elementary hardware operation that can be carried out in one clock cycle.
  - Register transfer operations, arithmetic and logic operations, etc.
- Thus a single machine instruction may take one or more CPU cycles to complete.
  - We can characterize an instruction by *Cycles Per Instruction* (CPI).
- Average CPI of a program:
  - Average CPI of all instructions executed in the program on a given processor.
  - Different instructions can have different CPIs.

- For a given program compiled to run on a specific machine, we can define the following parameters:
  a) The total number of instructions executed or *instruction count* (IC).
  b) The average number of *cycles per instruction* (CPI).
  c) *Clock cycle time* (C) of the machine.
- The total execution time can be computed as:

  Execution Time XT = IC x CPI x C

- How do we evaluate and compare the performances of several machines?

By Measuring the Execution Times

- One of the easiest methods to make the comparison.
- We measure the execution times of a program on two machines (A and B), as XT_A and XT_B.
- Performance can be defined as the reciprocal of execution time:

$$Perf_A = 1 / XT_A$$

$$Perf_B = 1 / XT_B$$

- We can estimate the speedup of machine A over machine B as:

$$Speedup = Perf_A / Perf_B = XT_B / XT_A$$

- **An example:**

  A program is run on three different machines A, B and C and execution times of 10, 25 and 75 are noted.

  - A is 2.5 times faster than B
  - A is 7.5 times faster than C
  - B is 3.0 times faster than C

- Simple for one program. But the main challenge is to extend the comparison when we have a set of programs.
  - Shall be discussed later.

Example 1

- A program is running on a machine with the following parameters:
  - Total number of instructions executed = 50,000,000
  - Average CPI for the program = 2.7
  - CPU clock rate = 2.0 GHz (i.e. C = 0.5 x 10^{-9} sec)
- Execution time of the program:

  XT = 50,000,000 x 2.7 x 0.5 x 10^{-9} = 0.0675 sec

XT = IC x CPI x C

Machine Cycles in 8085 Microprocessor

1. **Opcode Fetch Cycle (4T - 7T)**: Finds the type of instruction to be executed.
2. **Memory Read Cycle (3T)**: Reads an 8-bit data from a specified memory address.
3. **Memory Write Cycle (3T)**: Writes an 8-bit data into a specified memory address.
4. **I/O Read Cycle (3T)**: Reads an 8-bit data byte from an input port into the Accumulator.
5. **I/O Write Cycle (3T)**: Sends an 8-bit data byte from the Accumulator to an output port.
6. **Interrupt Acknowledge Cycle**: Responds to an active interrupt request.
7. **Bus Idle / Halt Cycle**: Halting requires 1T state.

Consider an Assembly Program that adds two 8-bit numbers stored at memory locations 2000H and 2001H, then stores the result at 2002H.

Instruction	**Machine Cycles Breakdown**	**T-states**
`LDA 2000H`	Opcode Fetch (4T) + Memory Read (3T) + Memory Read (3T) + Memory Read (3T)	13 T
`MOV B, A`	Opcode Fetch (4T)	4 T
`LDA 2001H`	Opcode Fetch (4T) + Memory Read (3T) + Memory Read (3T) + Memory Read (3T)	13 T
`ADD B`	Opcode Fetch (4T)	4 T
`STA 2002H`	Opcode Fetch (4T) + Memory Read (3T) + Memory Read (3T) + Memory Write (3T)	13 T
`HLT`	Opcode Fetch (4T) + Halt (1T)	5 T

- Total T-States: = 13T + 4T + 13T + 4T + 13T + 5T = 52T
- One T-State in 8085 (3MHz): = 1/(3 * 10^6) = 0.33 µs
- Total Execution Time: = 52 * 0.33 = 17.33 µs

Factors Affecting Performance

	C	CPI	IC
Hardware Technology (VLSI)	X		
Hardware Technology (Organization)	X	X	
Instruction set architecture		X	X
Compiler technology		X	X
Program		X	X

- IC depends on:
  - Program used, compiler, ISA
- CPI depends on:
  - Program used, compiler, ISA, CPU organization
- C depends on:
  - Technology used to implement the CPU
- Unfortunately, it is very difficult to change one parameter in complete isolation from the others.
  - Basic technologies are interdependent.

- A tradeoff:
  - *RISC*: increases number of instructions/program, but decreases CPI and clock cycle time because the instructions and hence the implementations are simple.
  - *CISC*: decreases number of instructions/program, but increases CPI and clock cycle time because many instructions are more complex.
- Overall, it has been found through experimentation that RISC architecture gives better performance.

Example 2

- Suppose that a machine A executes a program with an average CPI of 2.3.

  Consider another machine B (with the same instruction set and a better compiler) that executes the same program with 20% less instructions and with a CPI of 1.7 at 1.2 GHz.

  What should be the clock rate of A so that the two machines have the same performance?

We must have: $IC_A \times CPI_A \times C_A = IC_B \times CPI_B \times C_B$

Hence: $IC_A \times 2.3 \times C_A = 0.80 \times IC_A \times 1.7 \times (1 / (1.2 \times 10^9))$

We get: $C_A = 0.49 \times 10^{-9}$ sec

Thus, clock rate of A $= 1 / C_A = 2.04$ GHz

Example 3

- Consider the earlier example with IC = 50,000,000; average CPI = 2.7, and clock rate = 2.0 GHz.

  Suppose we use a new compiler on the same program, for which:

  - New IC = 40,000,000
  - New CPI = 3.0 (i.e. the new compiler is using more complex instructions)
  - Also we have a faster CPU implementation, with clock rate = 2.4 GHz.

$$\text{Speedup} = XT_{old} / XT_{new}$$
$$= (50{,}000{,}000 \times 2.7 \times 0.5 \times 10^{-9}) / (40{,}000{,}000 \times 3.0 \times 0.4167 \times 10^{-9})$$
$$= 1.35 \rightarrow \textit{35\% faster}$$

Instruction Types and CPI

- Consider a program executing on a processor, with *n* types or classes of instructions (like, load, store, ALU, branch, etc.).

  IC_i = number of instructions of type *i* executed

  CPI_i = cycles per instruction for type *i*

- The following expressions follow.

$$\text{CPU clock cycles} = \sum_{i=1}^{n} (IC_i \times CPI_i)$$

$$\text{Instruction Count (IC)} = \sum_{i=1}^{n} IC_i$$

$$\text{CPI} = \frac{\sum_{i=1}^{n} (IC_i \times CPI_i)}{IC} = \sum_{i=1}^{n} \left(\frac{IC_i}{IC} \times CPI_i \right)$$

Example 4

- Consider an implementation of a ISA where the instructions can be classified into four types, with CPI values of 1, 2, 3 and 4 respectively.

  Two code sequences have the following instruction counts:

Code Sequence	IC_{Type1}	IC_{Type2}	IC_{Type3}	IC_{Type4}
CS-1	20	15	5	2
CS-2	10	12	10	4

CPU cycles for CS-1: 20x1 + 15x2 + 5x3 + 2x4 = 73

CPI for CS-1: 73 / 42 = 1.74

CPU cycles for CS-2: 10x1 + 12x2 + 10x3 + 4x4 = 80

CPI for CS-2: 80 / 36 = 2.22

Instruction Frequency and CPI

- CPI can also be expressed in terms of the frequencies of the various instruction types that are executed in a program.
  - F_i denotes the frequency of execution of instruction type *i*.

$$\text{CPI} = \frac{\sum_{i=1}^{n}(IC_i \times CPI_i)}{IC} = \sum_{i=1}^{n}\left(\frac{IC_i}{IC} \times CPI_i\right)$$

$$F_i = \frac{IC_i}{IC}$$

$$\text{CPI} = \sum_{i=1}^{n}(F_i \times CPI_i)$$

Example 5

- Suppose for an implementation of a RISC ISA there are four instruction types, with their frequency of occurrence (for a typical mix of programs) and CPI as shown in the table below.

Type	Frequency	CPI
Load	20 %	4
Store	8 %	3
ALU	60 %	1
Branch	12 %	2

$$\text{CPI} = \sum_{i=1}^{n} (F_i \times CPI_i)$$

CPI = (0.20 x 4) + (0.08 x 3) + (0.60 x 1)
+ (0.12 x 2)
= 1.88

Example 6

- Suppose that a program is running on a machine with the following instruction types, CPI values, and the frequencies of occurrence.

  The CPU designer gives two options: (a) reduce CPI of instruction type A to 1.1, and (b) reduce CPI of instruction type B to 1.6. Which one is better?

Type	CPI	Frequency
A	1.3	60 %
B	2.2	10 %
C	2.0	30 %

Average CPI for (a): 0.60 x 1.1 + 0.10 x 2.2 + 0.30 x 2.0
= 1.48

Average CPI for (b): 0.60 x 1.3 + 0.10 x 1.6 + 0.30 x 2.0
= 1.54

Option (a) is better

Introduction

Gene Amadahl

- Amadahl's law was established in 1967 by Gene Amadahl.
- Basically provides an understanding on scaling, limitations and economics of parallel computing.
- Forms the basis for quantitative principles in computer system design.
  - Can be applied to other application domains as well.

What is Amadahl's Law?

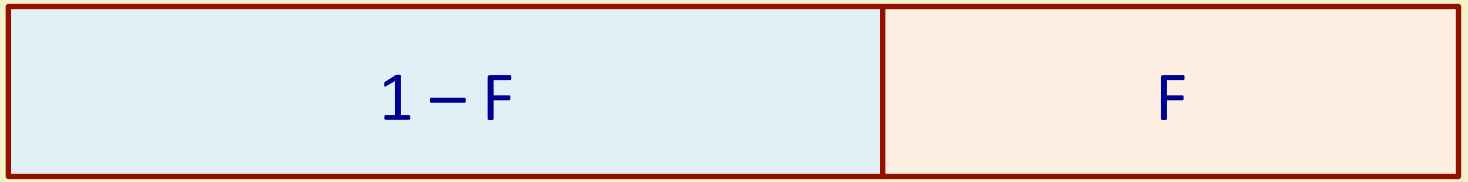
- It can be used to find the maximum expected improvement of an overall system when only *part of the system* is improved.
- It basically states that the performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used.
- Very useful to check whether any proposed improvement can provide expected return.
  - Used by computer designers to enhance only those architectural features that result in reasonable performance improvement.
  - Referred to as *quantitative principles in design*.

- Amadahl's law demonstrates the *law of diminishing returns*.
- An example:
  - Suppose we are improving a part of the computer system that affects only 25% of the overall task.
  - The improvement can be *very little* or *extremely large*.
  - With "*infinite*" speedup, the 25% of the task can be done in "*zero*" time.
  - Maximum possible speedup = $XT_{orig} / XT_{new} = 1 / (1 - 0.25) = 1.33$

We can never get a speedup of more than 1.33

- Amadahl's law concerns the speedup achievable from an improvement in computation that affects a fraction *F* of the computation, where the improvement has a speedup of *S*.

Before improvement



After improvement

- Execution time before improvement: $(1 - F) + F = 1$
- Execution time after improvement: $(1 - F) + F / S$
- Speedup obtained:

$$\text{Speedup} = \frac{1}{(1 - F) + F / S}$$

- As $S \rightarrow \infty$, $Speedup \rightarrow 1 / (1 - F)$
  - The fraction F limits the maximum speedup that can be obtained.

- Illustration of law of diminishing returns:
  - Let *F = 0.25*.
  - The table shows the speedup (*= 1 / (1 – F + F / S*) for various values of *S*.

1 / (1 – 0.25) = 1.33

S	Speedup
1	1.00
2	1.14
5	1.25
10	1.29

S	Speedup
50	1.32
100	1.33
1000	1.33
100,000	1.33

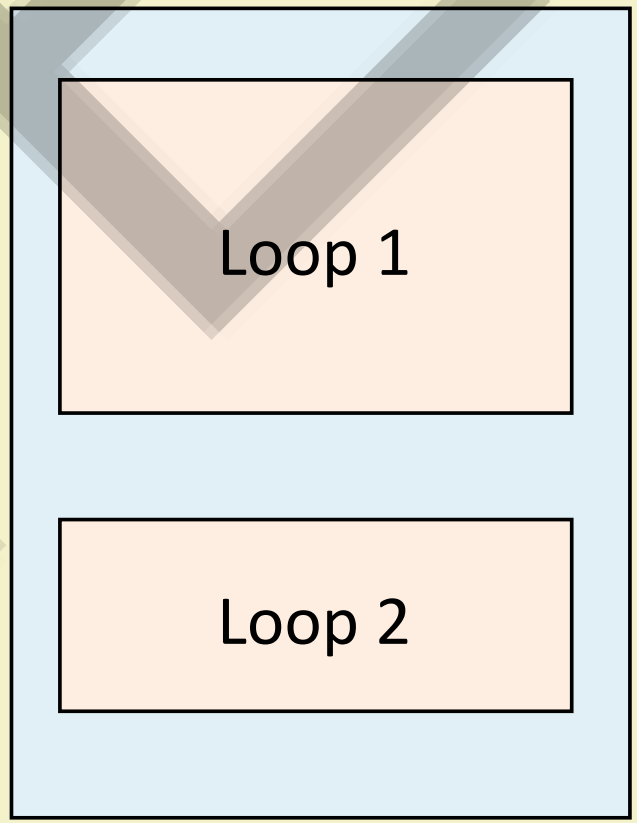
- Illustration of law of diminishing returns:
  - Let *F = 0.75*.
  - The table shows the speedup for various values of *S*.

1 / (1 – 0.75) = 4.00

S	Speedup
1	1.00
2	1.60
5	2.50
10	3.08

S	Speedup
50	3.77
100	3.88
1000	3.99
100,000	4.00

Design Alternative using Amadahl's law



500 lines 10% of total execution time

20 lines 90% of total execution time

- Some examples:
  - We make 10% of a program 90X faster, speedup = 1 / (0.9 + 0.1 / 90) = 1.11
  - We make 90% of a program 10X faster, speedup = 1 / (0.1 + 0.9 / 10) = 5.26
  - We make 25% of a program 25X faster, speedup = 1 / (0.75 + 0.25 / 25) = 1.32
  - We make 50% of a program 20X faster, speedup = 1 / (0.5 + 0.5 / 20) = 1.90
  - We make 90% of a program 50X faster, speedup = 1 / (0.1 + 0.9 / 50) = 8.47

Example 1

- Suppose we are running a set of programs on a RISC processor, for which the following instruction mix is observed:

Operation	Frequency	CPI_i	$W_i * CPI_i$	% Time
Load	20 %	5	1.00	0.48
Store	8 %	3	0.24	0.12
ALU	60 %	1	0.60	0.29
Branch	12 %	2	0.24	0.11

CPI = 2.08

1 / 2.08

We carry out a design enhancement by which the CPI of Load instructions reduces from 5 to 2. What will be the overall performance improvement?

Fraction enhanced $F = 0.48$

Fraction unaffected $1 - F = 1 - 0.48 = 0.52$

Enhancement factor $S = 5 / 2 = 2.5$

Therefore, speedup is

$$\frac{1}{(1 - F) + F / S} = \frac{1}{0.52 + 0.48 / 2.5} = 1.40$$

- Alternate way of calculation:
  - Old CPI = 2.08
  - New CPI = 0.20 * **2** + 0.08 * 3 + 0.60 * 1 + 0.12 * 2 = 1.48

$$Speedup = \frac{XT_{orig}}{XT_{new}} = \frac{IC * CPI_{old} * C}{IC * CPI_{new} * C}$$
$$= \frac{CPI_{old}}{CPI_{new}} = \frac{2.08}{1.48} = 1.40$$

Example 2

- The execution time of a program on a machine is found to be 50 seconds, out of which 42 seconds is consumed by multiply operations. It is required to make the program run 5 times faster. By how much must the speed of the multiplier be improved?
  - Here, $F = 42 / 50 = 0.84$
  - According to Amadahl's law,

$$5 = 1 / (0.16 + 0.84 / S)$$

$$\text{or, } 0.80 + 4.2 / S = 1$$

$$\text{or, } S = 21$$

Example 2a

- The execution time of a program on a machine is found to be 50 seconds, out of which 42 seconds is consumed by multiply operations. It is required to make the program run **8** times faster. By how much must the speed of the multiplier be improved?
  - Here, F = 42 / 50 = 0.84
  - According to Amadahl's law,

$$8 = 1 / (0.16 + 0.84 / S)$$

$$\text{or, } 1.28 + 6.72 / S = 1$$

$$\text{or, } S = -24$$

No amount to speed improvement in the multiplier can achieve this.

Maximum speedup achievable:

$$1 / (1 - F) = 6.25$$

Example 3

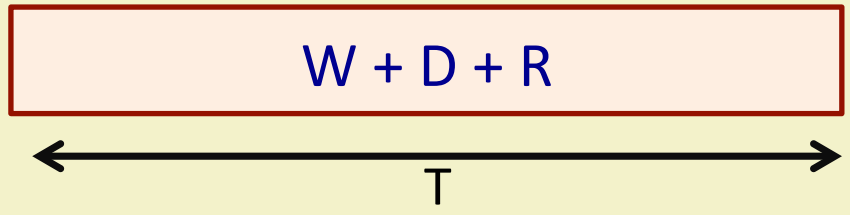
- Suppose we plan to upgrade the processor of a web server. The CPU is 30 times faster on search queries than the old processor. The old processor is busy with search queries 80% of the time. Estimate the speedup obtained by the upgrade.
  - Here, F = 0.80 and S = 30
  - Thus, speedup = 1 / (0.20 + 0.80 / 30) = 4.41

What is Pipelining?

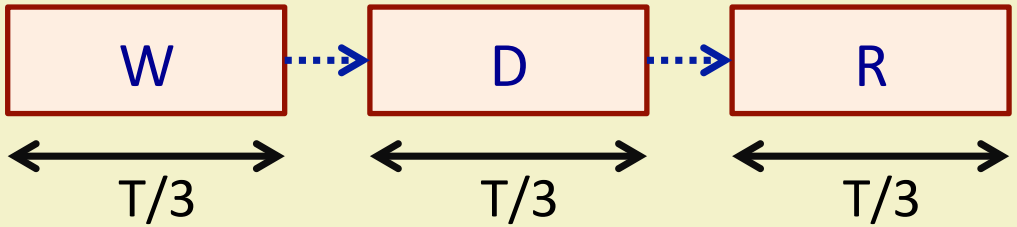
- A mechanism for overlapped execution of several input sets by partitioning some computation into a set of *k* sub-computations (or stages).
  - Very nominal increase in the cost of implementation.
  - Very significant speedup (ideally, *k*).
- Where are pipelining used in a computer system?
  - **Instruction execution**: Several instructions executed in some sequence.
  - **Arithmetic computation**: Same operation carried out on several data sets.
  - **Memory access**: Several memory accesses to consecutive locations are made.

A Real-life Example

- Suppose you have built a machine *M* that can wash (*W*), dry (*D*), and iron (*R*) clothes, one cloth at a time.
  - Total time required is *T*.
- As an alternative, we split the machine into three smaller machines M_W, M_D and M_R, which can perform the specific task only.
  - Time required by each of the smaller machines is *T/3* (say).

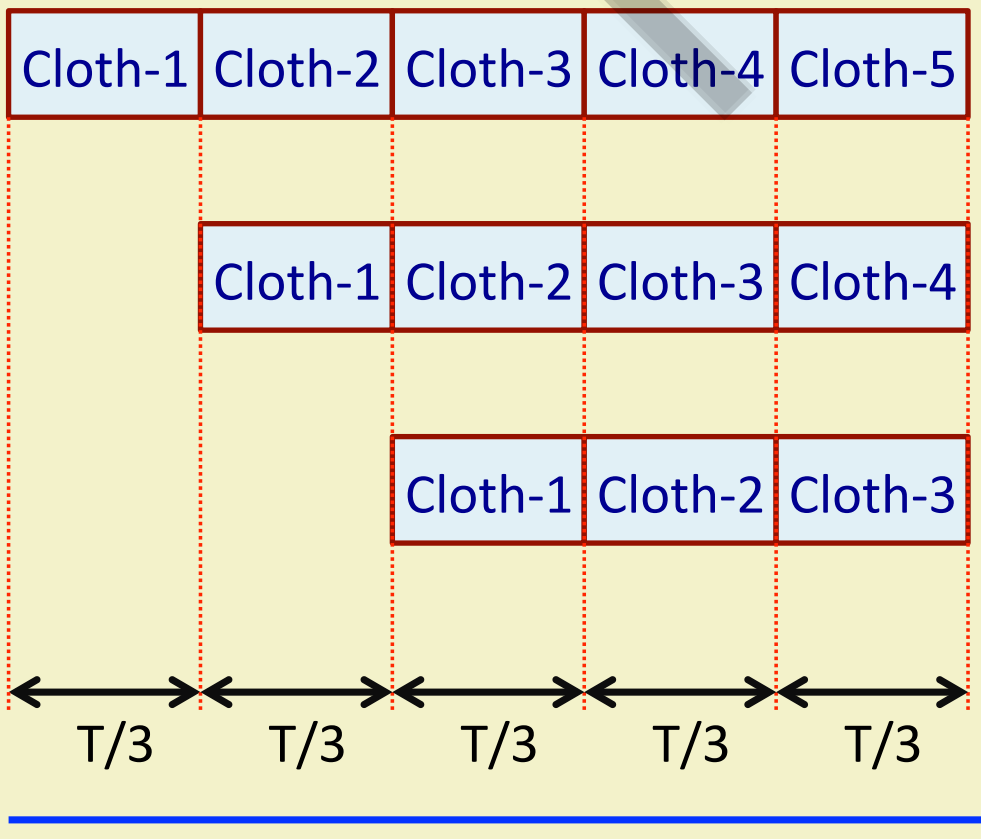


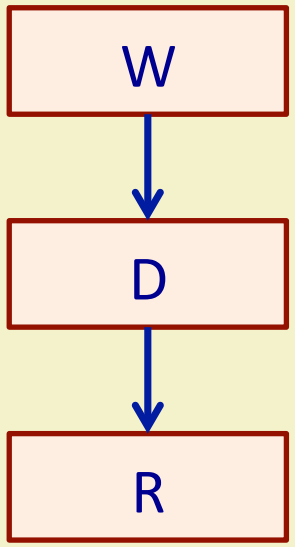
For N clothes, time $T_1 = N.T$



For N clothes, time $T_3 = (2 + N).T/3$

How does the pipeline work?





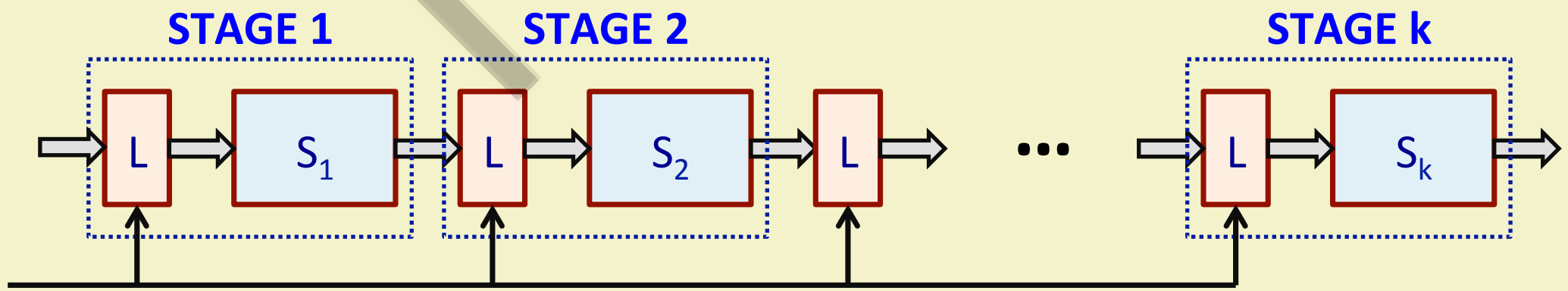
Finishing times:

- Cloth-1 – 3.T/3
- Cloth-2 – 4.T/3
- Cloth-3 – 5.T/3
- …
- Cloth-N – (2 + N).T/3

Extending the Concept to Processor Pipeline

- The same concept can be extended to hardware pipelines.
- Suppose we want to attain *k* times speedup for some computation.
  - **Alternative 1**: Replicate the hardware *k* times → cost also goes up *k* times.
  - **Alternative 2**: Split the computation into *k* stages → very nominal cost increase.
- Need for buffering:
  - In the washing example, we need a tray between machines (W & D, and D & R) to keep the cloth temporarily before it is accepted by the next machine.
  - Similarly in hardware pipeline, we need a *latch* between successive stages to hold the intermediate results temporarily.

Model of a Synchronous k-stage Pipeline



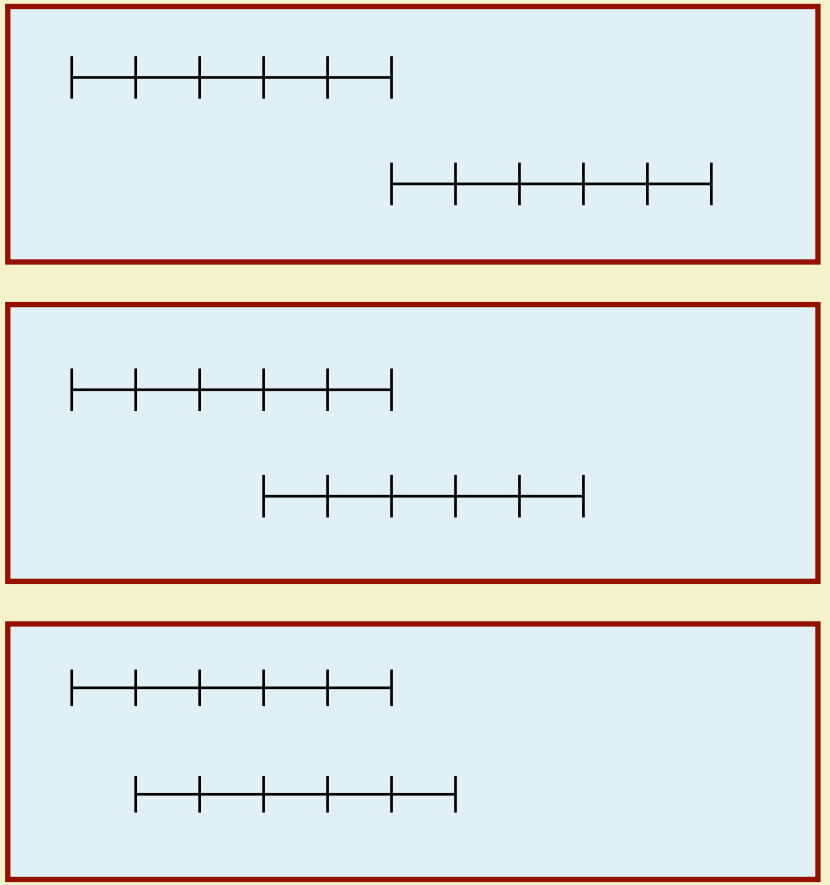
- The latches are made with master-slave flip-flops, and serve the purpose of isolating inputs from outputs.
- The pipeline stages are typically combinational circuits.
- When *Clock* is applied, all latches transfer data to the next stage simultaneously.

Types of Pipelined Processors

- Can be classified based on various parameters:
  a) Degree of overlap
     - Serial, overlapped or pipelined
  b) Depth of the pipeline
     - Shallow or Deep
  c) Structure of the pipeline
     - Linear or Non-linear
  d) How the operations are scheduled in the pipeline?
     - Static or Dynamic

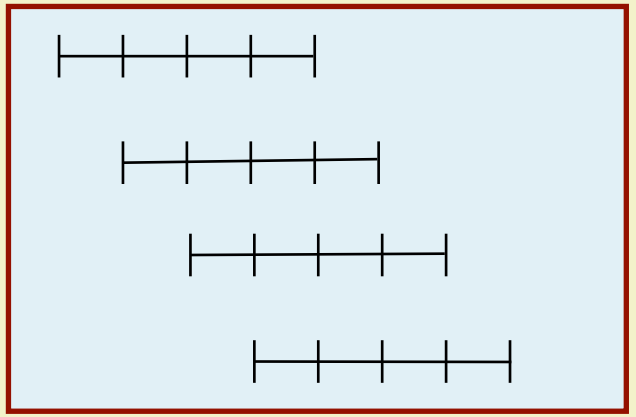
(a) Degree of Overlap

- Serial
  - The next operation can start only after the previous operation finishes.
- Overlapped
  - There is some overlap between successive operations.
- Pipelined
  - Fine-grain overlap between successive operations.

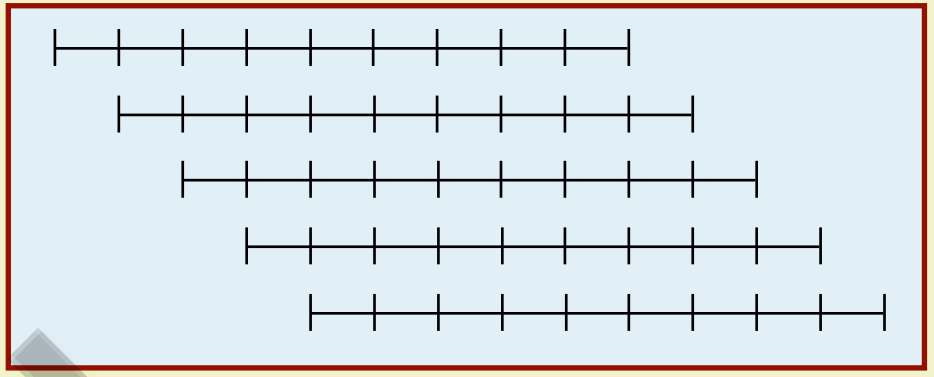


(b) Depth of the Pipeline

- Performance of a pipeline depends on the number of stages and how they can be utilized without conflict.
- Shallow pipeline is one with fewer number of stages.
  - Individual stages more complex.
- Deep pipeline is one with larger number of stages.
  - Individual stages simpler.



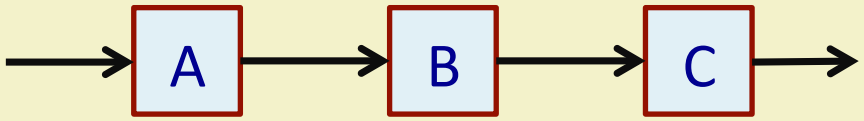
Shallow

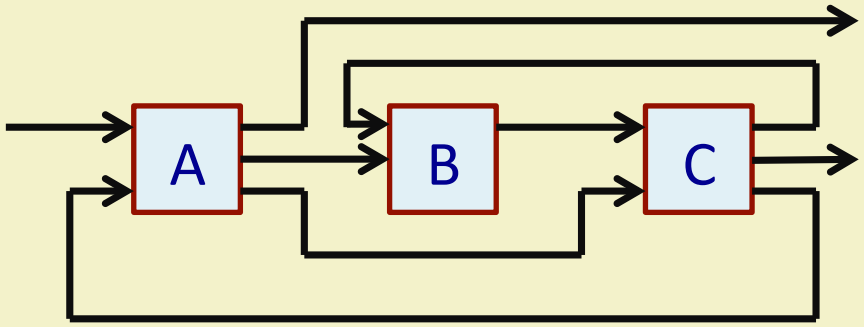


Deep

(c) Structure of the Pipeline

- **Linear Pipeline**: The stages that constitute the pipeline are executed one by one in sequence (say, from left to right).
- **Non-linear Pipeline**: The stages may not execute in a linear sequence (say, a stage may execute more than once for a given data set).





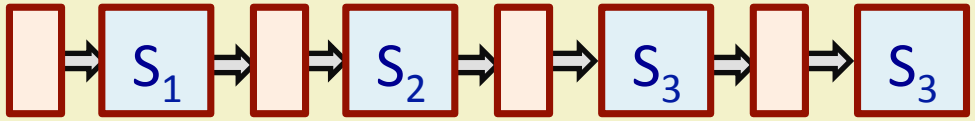
A possible sequence: A, B, C, B, C, A, C, A

(d) Scheduling Alternatives

- Static Pipeline:
  - Same sequence of pipeline stages are executed for all data / instructions.
  - If one data / instruction stalls, all subsequent ones also gets delayed.
- Dynamic Pipeline:
  - Can be reconfigured to perform variable functions at different times.
  - Allows feedforward and feedback connections between stages.

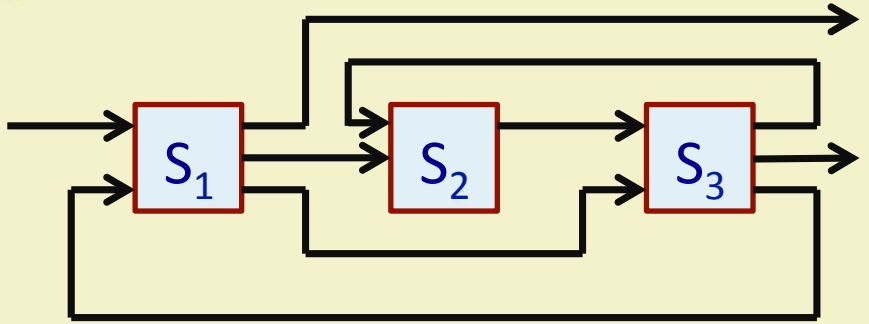
Reservation Table

- The *Reservation Table* is a data structure that represents the utilization pattern of successive stages in a synchronous pipeline.
  - Basically a space-time diagram of the pipeline that shows precedence relationships among pipeline stages.
    - X-axis shows the time steps
    - Y-axis shows the stages
  - Number of columns give evaluation time.
  - The reservation table for a 4-stage linear pipeline is shown.



	1	2	3	4
S_1	X			
S_2		X		
S_3			X	
S_4				X

- Reservation table for a 3-stage dynamic multi-function pipeline is shown.
  - Contains feedforward and feedback connections.
  - Two functions X and Y.
- Some characteristics:
  - *Multiple X's in a row* :: repeated use of the same stage in different cycles.
  - *Contiguous X's in a row* :: extended use of a stage over more than one cycles.
  - *Multiple X's in a column* :: multiple stages are used in parallel during a clock cycle.



	1	2	3	4	5	6	7	8
S_1	X					X		X
S_2		X		X				
S_3			X		X		X	

	1	2	3	4	5	6
S_1	Y				Y	
S_2			Y			
S_3		Y		Y		Y

Speedup and Efficiency

Some notations:

τ :: clock period of the pipeline

t_i :: time delay of the circuitry in stage S_i

d_L :: delay of a latch

Maximum stage delay $\tau_m = \max \{t_i\}$

Thus, $\tau = \tau_m + d_L$

Pipeline frequency $f = 1 / \tau$

– If one result is expected to come out of the pipeline every clock cycle, f will represent the maximum throughput of the pipeline.

- The total time to process *N* data sets is given by

$$T_k = [(k - 1) + N].\tau$$

(k – 1) τ time required to fill the pipeline

1 result every τ time after that → total N.τ

- For an equivalent non-pipelined processor (i.e. one stage), the total time is

$$T_1 = N.k.\tau$$

(ignoring the latch overheads)

- Speedup of the k-stage pipeline over the equivalent non-pipelined processor:

$$S_k = \frac{T_1}{T_k} = \frac{N.k.\tau}{k.\tau + (N - 1).\tau} = \frac{N.k}{k + (N - 1)}$$

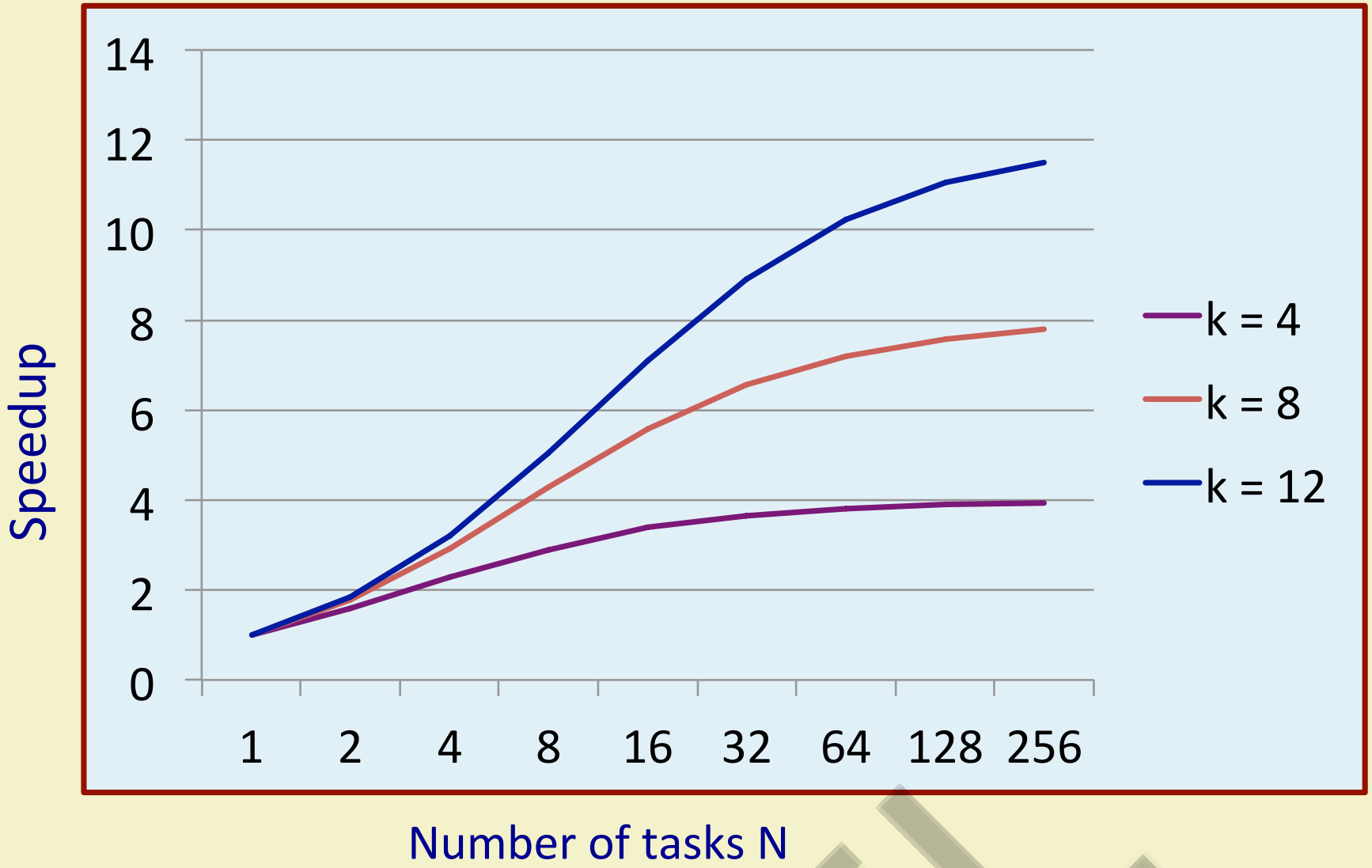
As $N \rightarrow \infty$, $S_k \rightarrow k$

- Pipeline efficiency:
  - How close is the performance to its ideal value?

$$E_k = \frac{S_k}{k} = \frac{N}{k + (N - 1)}$$

- Pipeline throughput:
  - Number of operations completed per unit time.

$$H_k = \frac{N}{T_k} = \frac{N}{[k + (N - 1)].\tau}$$


14
12
10
8
6
4
2
0
Speedup
1
2
4
8
16
32
64
128
256
Number of tasks N
k = 4
k = 8
k = 12

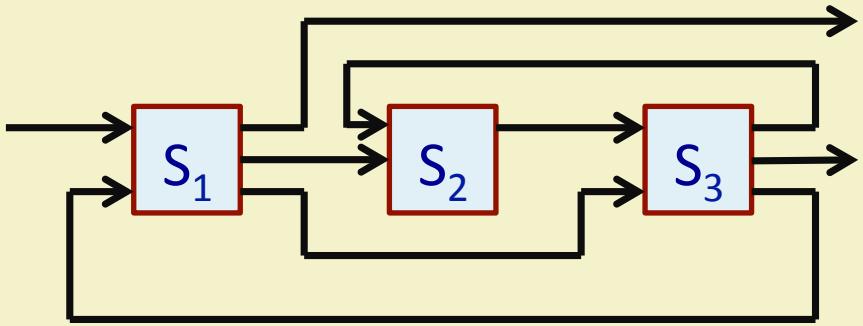
Clock Skew / Jitter / Setup time

- The minimum clock period of the pipeline must satisfy the inequality:

$$\tau \geq t_{skew+jitter} + t_{logic+setup}$$

- Definitions:
  - *Skew*: Maximum delay difference between the arrival of clock signals at the stage latches.
  - *Jitter*: Maximum delay difference between the arrival of clock signal at the same latch.
  - *Logic delay*: Maximum delay of the slowest stage in the pipeline.
  - *Setup time*: Minimum time a signal needs to be stable at the input of a latch before it can be captured.

Scheduling of Non-linear Pipelines



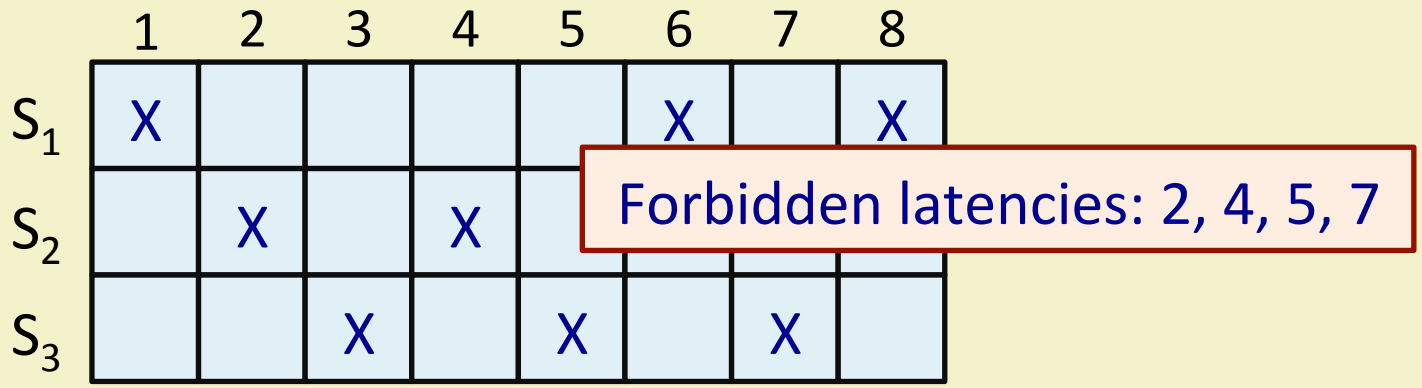
	1	2	3	4	5	6	7	8
S_1	X					X		X
S_2		X		X				
S_3			X		X		X	

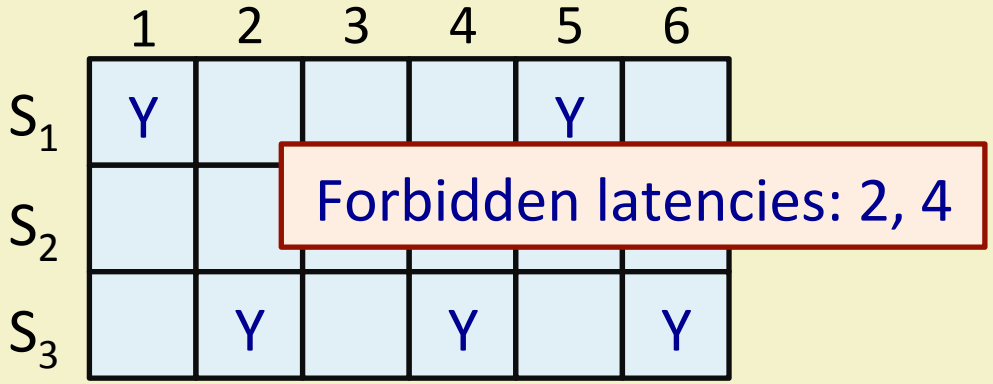
Two operations X and Y

- **X: 8 time steps to complete**
- **Y: 6 time steps to complete**

	1	2	3	4	5	6
S_1	Y				Y	
S_2			Y			
S_3		Y		Y		Y

- Latency Analysis:
  - The number of time units between two initiations of a pipeline is called the *latency* between them.
  - Any attempt by two or more initiations to use the same pipeline stage at the same time will cause a *collision*.
  - The latencies that can cause collision are called *forbidden latencies*.
    - Distance between two X's in the same row of the reservation table.





– A *latency sequence* is a sequence of permissible non-forbidden latencies between successive task initiations.

– A *latency cycle* is a latency sequence that repeats the same subsequence.

Function X

- Forbidden latencies: 2, 4, 5, 7
- Possible latency cycles:
  - (1, 8) = 1, 8, 1, 8, … (average latency = 4.5)
  - (3) = 3, 3, 3, … (average latency = 3.0)
  - (6) = 6, 6, 6, … (average latency = 6.0)

Function Y

- Forbidden latencies: 2, 4
- Possible latency cycles:
  - (1, 5) = 1, 5, 1, 5, … (average latency = 3.0)
  - (3) = 3, 3, 3, … (average latency = 3.0)
  - (3, 5) = 3, 5, 3, 5, … (average latency = 4.0)

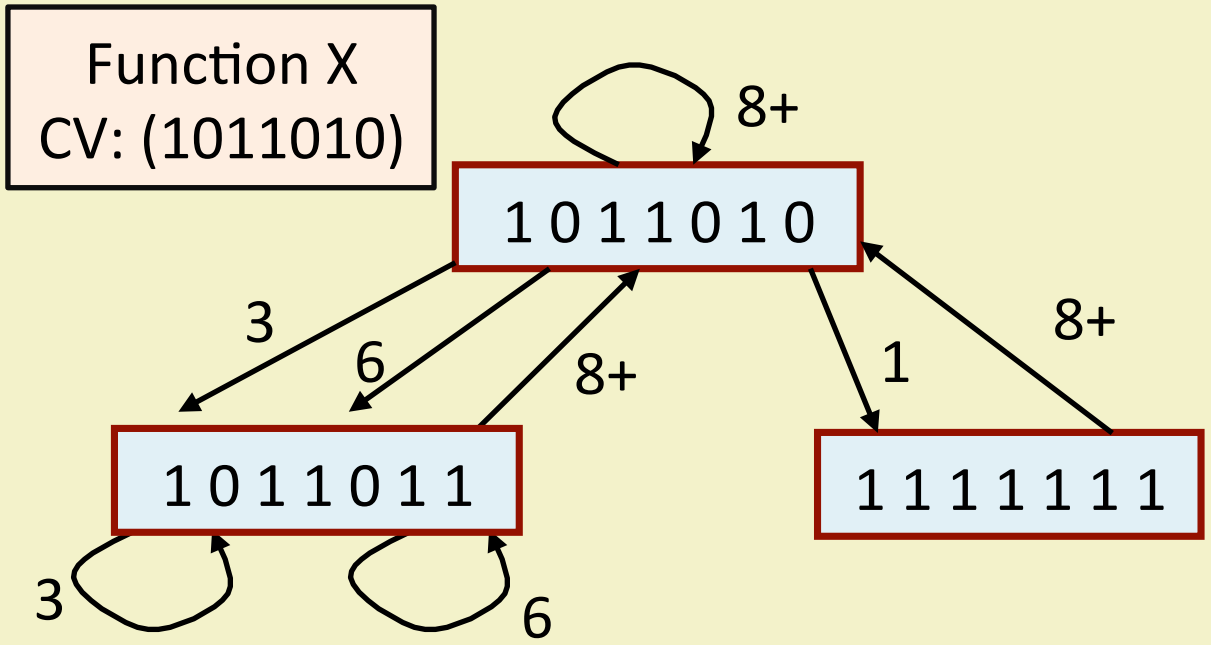
Constant Cycle

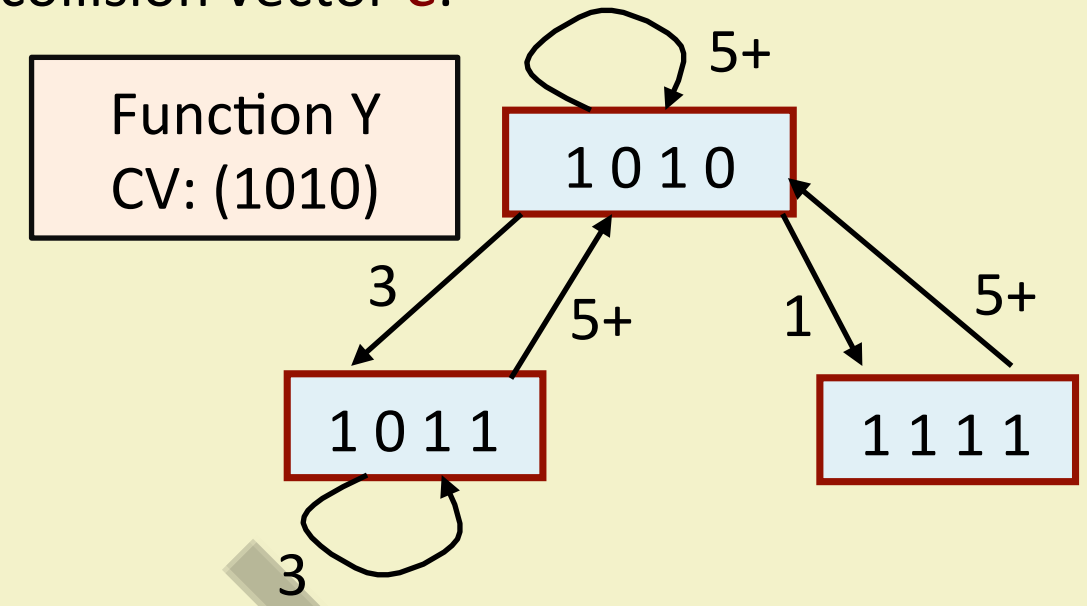
Collision Free Scheduling

- Main objective:
  - Obtain the shortest average latency between initiations without causing collisions.
- We define a *collision vector*.
  - If the reservation table has n columns, the maximum forbidden latency is $m \leq n-1$.
  - The permissible latencies p will satisfy: $1 \leq p \leq m-1$.
  - The collision vector is an m-bit binary vector $C = (C_m\ C_{m-1} \ldots C_2\ C_1)$, where $C_i = 1$ if latency i causes collision, and $C_i = 0$ otherwise.
  - C_m is always 1.

Function X: $C_X = (1011010)$
Function Y: $C_Y = (1010)$

- From the collision vector (CV), we can construct a *state diagram* specifying the permissible state transitions among successive initiations.
  - The collision vector corresponds to the initial state of the pipeline.
  - The next state at time *t+p* is obtained by shifting the present state *p*-bits to the right and OR-ing with the initial collision vector *C*.





- From the state diagram, we can determine latency cycles that result in *minimum average latency (MAL)*.
  - In a *simple cycle*, a state appears only once.
  - Some of the simple cycles are *greedy cycles*, which are formed only using outgoing edges with minimum latencies.

Function X:

- Simple cycles: (3), (6), (8), (1,8), (3,8), (6,8)
- Greedy cycles: (3), (1,8)

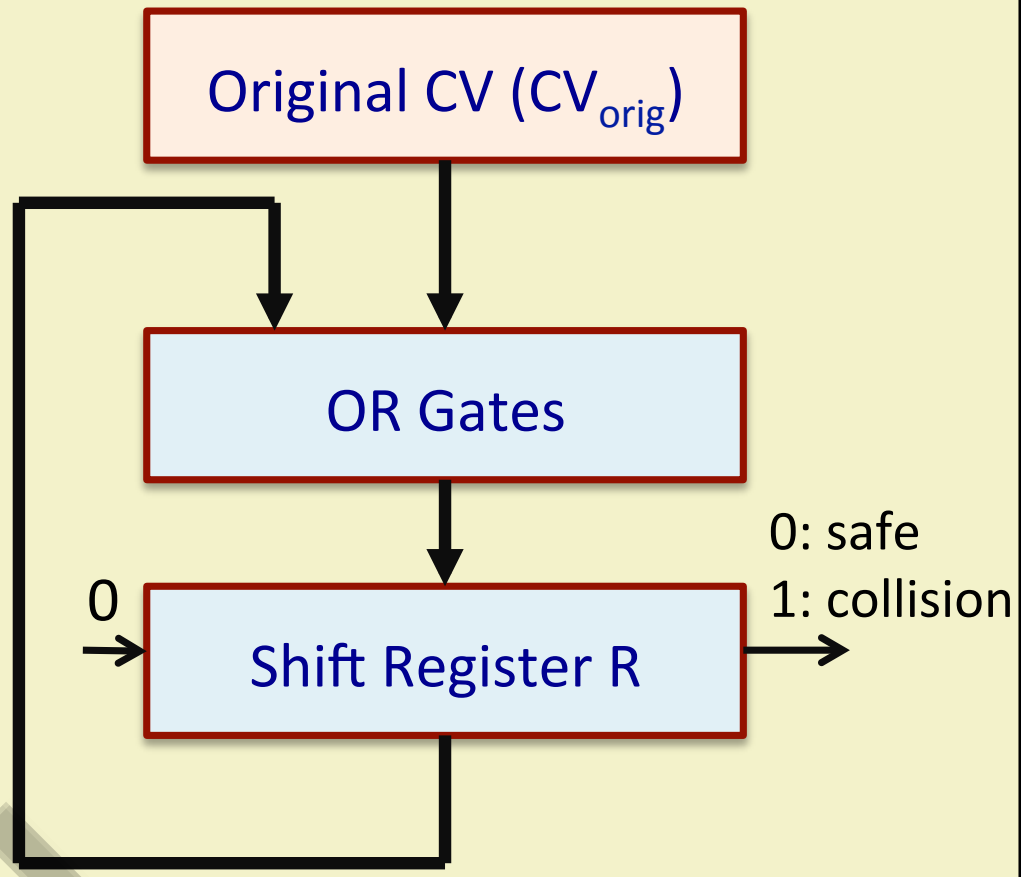
Function Y:

- Simple cycles: (3), (5), (1,5), (3,5)
- Greedy cycles: (3), (1,5)

MAL = 3 for both X and Y

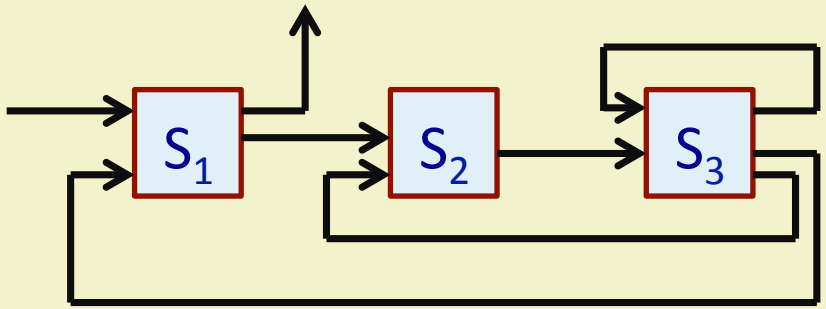
The Scheduling Algorithm

Load collision vector (CV) in a shift register R;
If (LSB of R is 1) then
    begin
        Do not initiate an operation;
        Shift R right by one position with 0 insertion;
    end
else
    begin
        Initiate an operation;
        Shift R right by one position with 0 insertion;
        R = R OR CV_{orig}; // Logical OR with original CV
    end

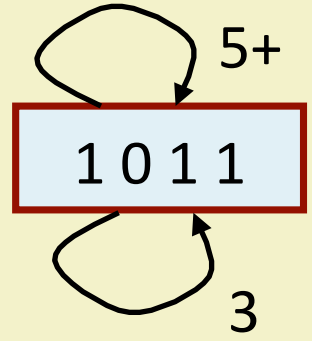


Optimizing a Pipeline Schedule

- We can insert non-compute (dummy) delay stages into the original pipeline.
  - This will modify the reservation table, resulting in a new collision vector.
  - Possibly a shorter MAL.

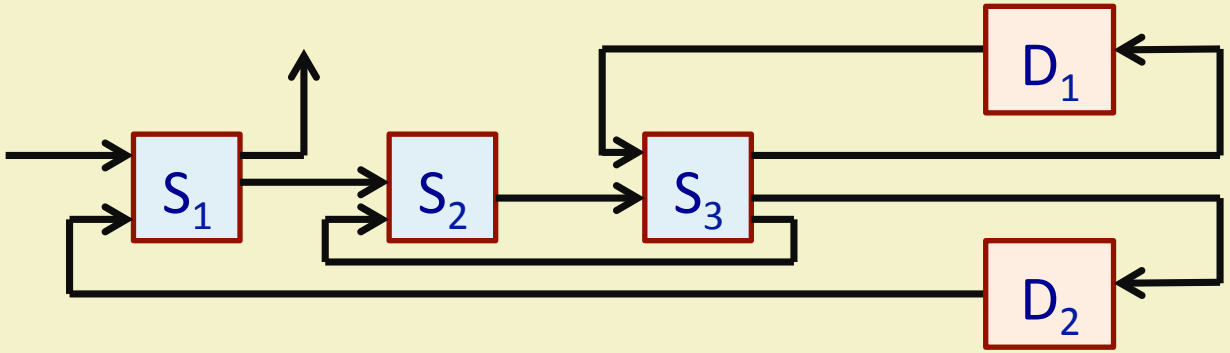


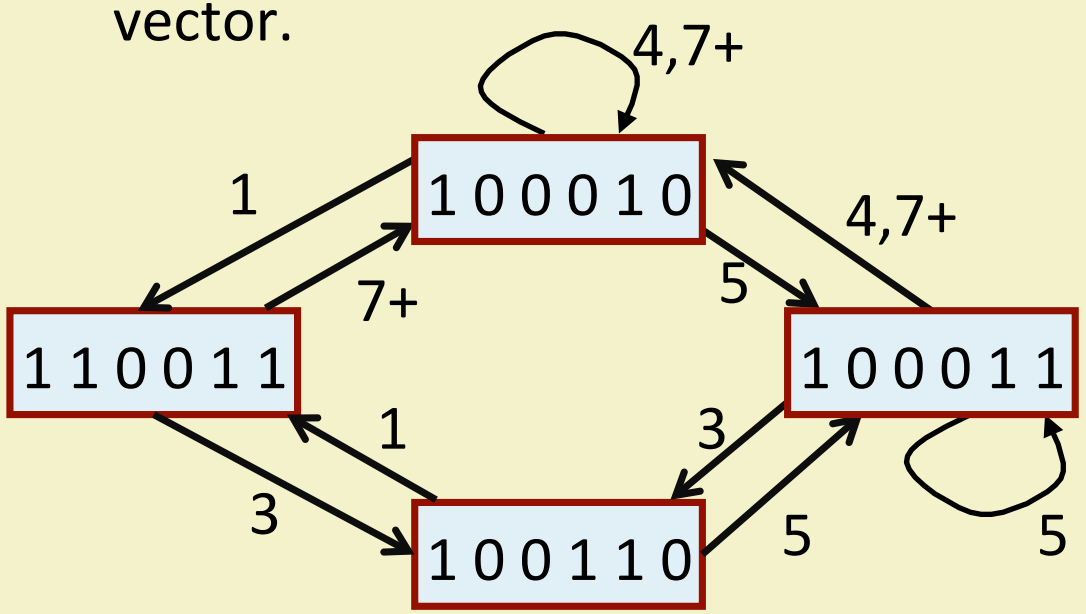
	1	2	3	4	5
S_1	X				X
S_2		X		X	
S_3			X	X	



MAL = 3

- Suppose we insert delay elements D_1 and D_2 as shown.
  - This will modify the reservation table, resulting in a new collision vector.





	1	2	3	4	5	6	7
S_1	X				→	→	X_2
S_2		X		X			
S_3			X	→	X_1		
D_1				X			
D_2						X	

MAL = 2 (for the greedy cycle (1,3))

Exercise 1

- For the following reservation tables,
  a) What are the forbidden latencies?
  b) Show the state transition diagram.
  c) List all the simple cycles and greedy cycles.
  d) Determine the optimal constant latency cycle, and the MAL.
  e) Determine the pipeline throughput, for τ = 20 ns.

	1	2	3	4
S_1	X			X
S_2		X		
S_3			X	

	1	2	3	4	5	6	7
S_1	X		X				X
S_2		X			X		
S_3				X		X	

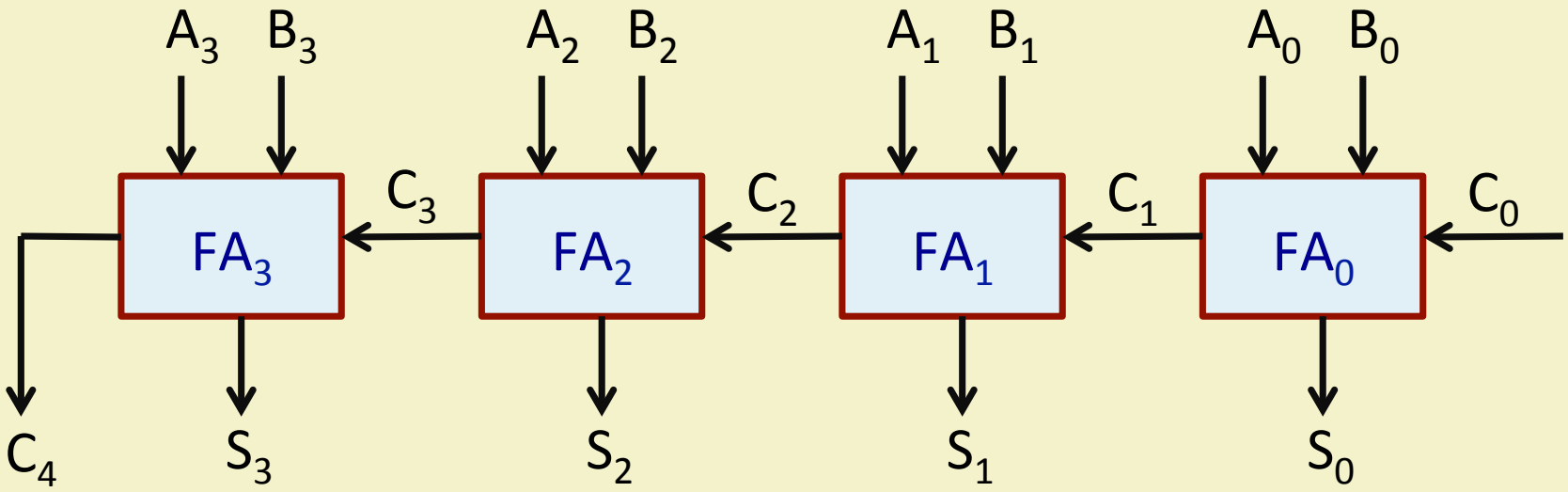
Exercise 2

- A non-pipelined processor X has a clock frequency of 250 MHz and an average CPI of 4. Processor Y, an improved version of X, is designed with a 5-stage linear instruction pipeline. However, due to latch delay and clock skew, the clock rate of Y is only 200 MHz.
  a) If a program consisting of 5000 instructions are executed on both processors, what will be the speedup of processor Y as compared to processor X?
  b) Calculate the MIPS rate of each processor during the execution of this particular program.

Fixed Point Addition Pipeline

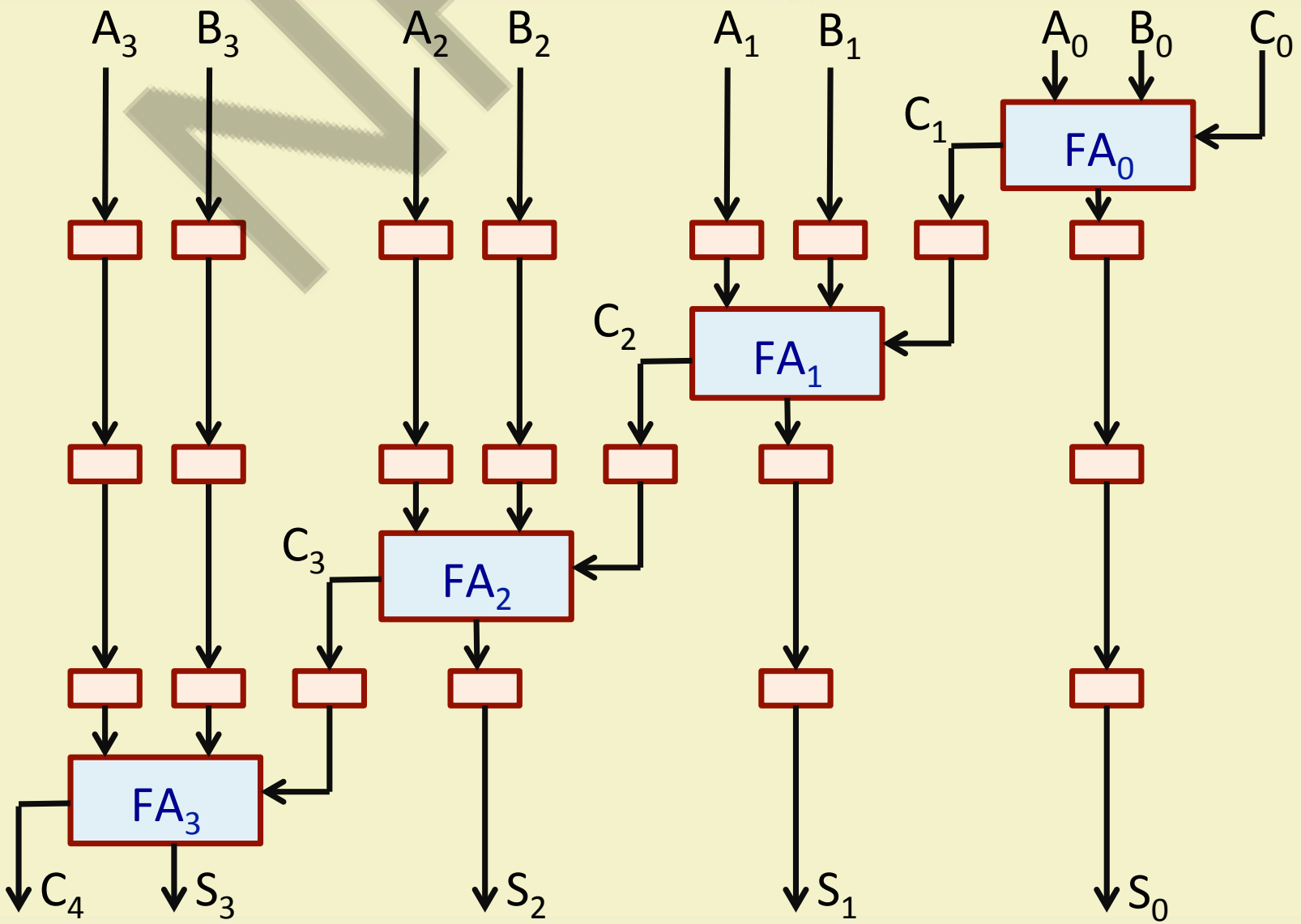
- We have seen how a ripple-carry adder works.
  - Rippling of the carries gives it a bad worst-case performance.
- We explore whether pipelining can improve the performance.
- *Assumption*: delay of a latch is comparable to the delay of a full adder.

A 4-bit Ripple Carry Adder



Worst-case delay ≈ 4 x (carry generation time in FA)

4-bit pipelined ripple-carry adder



1-bit latch

- Delay of a full adder = t_{FA}
- Delay of a 1-bit latch = t_L
- Clock period $T \geq (t_{FA} + t_L)$
- After the pipeline is full, one result (sum) is generated every time T.
  - Convenient for vector addition kind of applications.

```
for (i=0; i<10000; i++)
    a[i] = b[i] + c[i];
```

Overview of Memory Organization

- Memory is one of the most important sub-systems of a computer that determines the overall performance.
- Conceptual view of memory:
  - Array of storage locations, with each storage location having a unique address.
  - Each storage location can hold a fixed amount of information (multiple of bits, which is the basic unit of data storage).
- A memory system with *M* locations and *N* bits per location, is referred to as an *M x N* memory.
  - Both *M* and *N* are typically some powers of 2.
  - Example: 1024 x 8, 65536 x 32, etc.

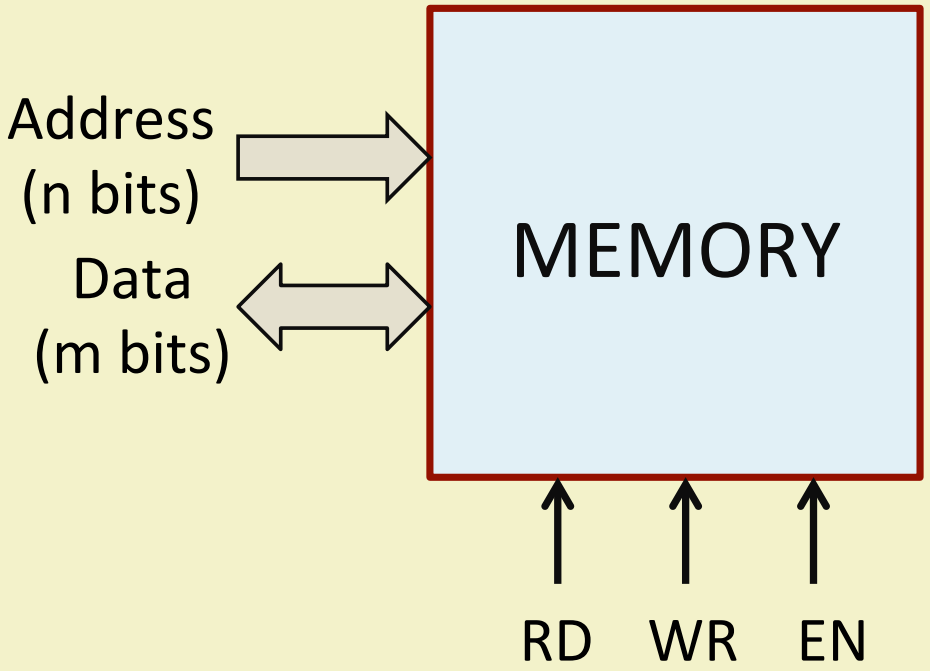
How is Memory Organized?

- Memory is often byte organized.
  - Every byte of the memory has a unique address.
- Multiple bytes of data can be accessed by an instruction.
  - Example: *Half-word* (2 bytes), *Word* (4 bytes), *Long Word* (8 bytes).
- For higher data transfer rate, memory is often organized such that multiple bytes can be read or written simultaneously.
  - Necessary to bridge the processor-memory speed gap.
  - Shall be discussed later in detail.

How do we Specify Memory Sizes?

Unit	Bytes	In Decimal
8 bits (B)	1 or 2^0	10^0
Kilobyte (KB)	1024 or 2^{10}	10^3
Megabyte (MB)	1,048,576 or 2^{20}	10^6
Gigabyte (GB)	1,073,741,824 or 2^{30}	10^9
Terabyte (TB)	1,099,511,627,776 or 2^{40}	10^{12}
Petabyte (PB)	2^{50}	10^{15}
Exabyte (EB)	2^{60}	10^{18}
Zettabyte (ZB)	2^{70}	10^{21}

- If there are *n* bits in the address, the maximum number of storage locations can be 2^n.
  - For n=8, 256 locations.
  - For n=16, 64K locations.
  - For n=20, 1M locations.
  - For n=32, 4G locations.
- Modern-day memory chips can store several Gigabits of data.
  - Dynamic RAM (DRAM).



Address	Contents
0000 0000	0000 0000 0000 0001
0000 0001	0000 0100 0101 0000
0000 0010	1010 1000 0000 0000
⋮	⋮
1111 1111	1011 0000 0000 1010

An example: 2^8 x 16 memory

Some Examples

1. A computer has 64 MB (megabytes) of byte-addressable memory. How many bits are needed in the memory address?
   - Address Space = 64 MB = 2^6 X 2^{20} B = 2^{26} B
   - If the memory is byte addressable, we need 26 bits of address.
2. A computer has 1 GB of memory. Each word in this computer is 32 bits. How many bits are needed to address any single word in memory?
   - Address Space = 1 GB = 2^{30} B
   - 1 word = 32 bits = 4 B
   - We have 2^{30} / 4 = 2^{28} words
   - Thus, we require 28 bits to address each word.

Classification of Memory Systems

a) Volatile versus Non-volatile:

- A *volatile* memory system is one where the stored data is lost when the power is switched off.
  - Examples: CMOS static memory, CMOS dynamic memory.
  - Dynamic memory in addition requires periodic refreshing.
- A *non-volatile* memory system is one where the stored data is retained even when the power is switched off.
  - Examples: Read-only memory, Magnetic disk, CDROM/DVD, Flash memory, Resistive memory.

b) Random-access versus Direct/Sequential access:

- A memory is said to be *random-access* when the read/write time is independent of the memory location being accessed.
  - Examples: CMOS memory (RAM and ROM).
- A memory is said to be *sequential access* when the stored data can only be accessed sequentially in a particular order.
  - Examples: Magnetic tape, Punched paper tape.
- A memory is said to be *direct or semi-random access* when part of the access is sequential and part is random.
  - Example: Magnetic disk.
  - We can directly go to a track after which access will be sequential.

c) Read-only versus Random-access:

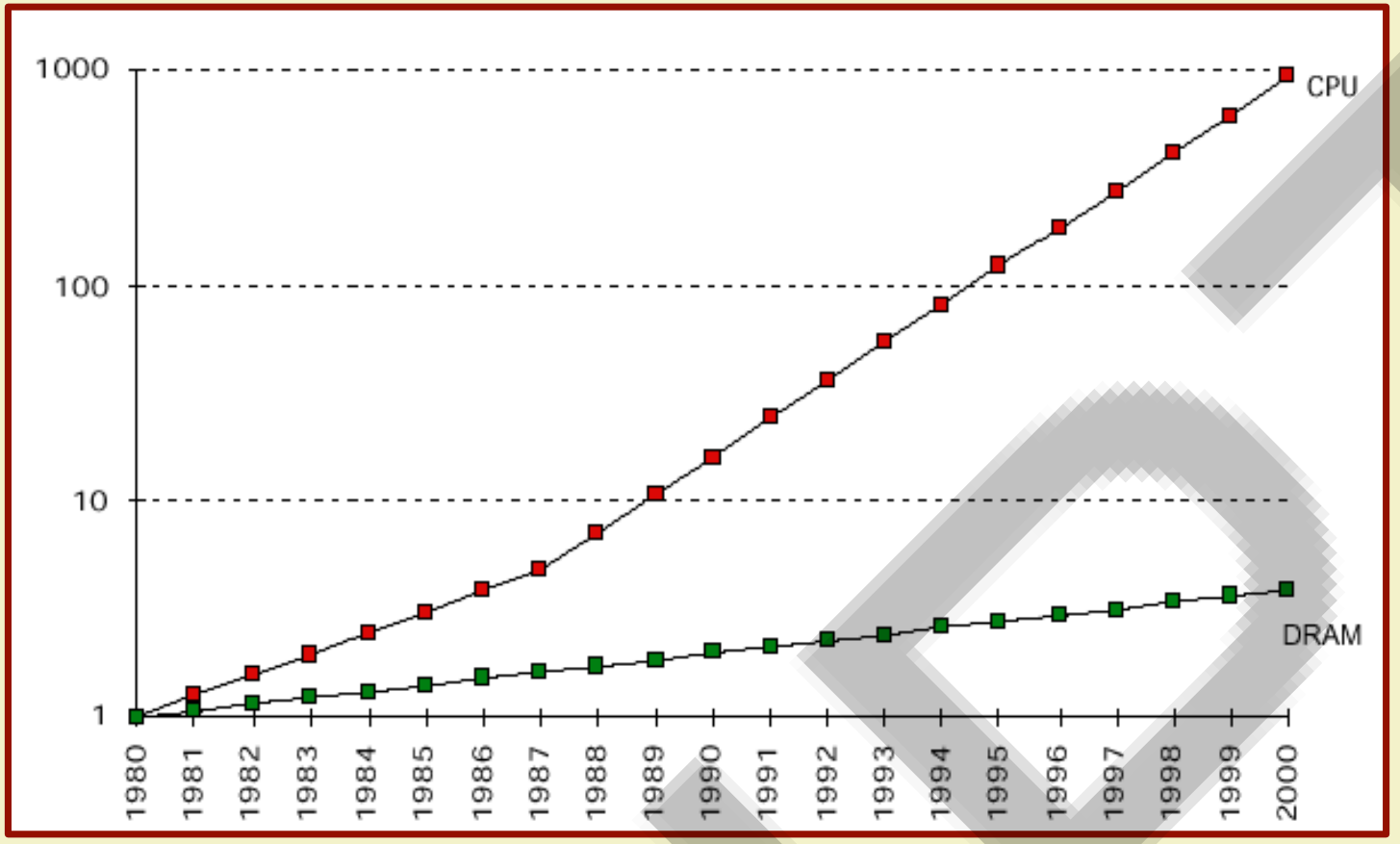
- *Read-only Memory* (ROM) is one where data once stored in permanent or semi-permanent.
  - Data written (programmed) during manufacture or in the laboratory.
  - Examples: ROM, PROM, EPROM, EEPROM.
- *Random Access Memory* (RAM) is one where data access time is the same independent of the location (address).
  - Used in main / cache memory systems.
  - Example: Static RAM (SRAM) → data once written are retained as long as power is on.
  - Example: Dynamic RAM (DRAM) → requires periodic refreshing even when power is on (data stored as charge on tiny capacitors).

Access Time, Latency and Bandwidth

- Terminologies used to measure speed of the memory system.
  a) Memory Access Time: Time between initiation of an operation (Read or Write) and completion of that operation.
  b) Latency: Initial delay from the initiation of an operation to the time the first data is available.
  c) Bandwidth: Maximum speed of data transfer in bytes per second.
- In modern memory organizations, every read request reads a block of words into some high-speed registers (LATENCY), from where data are supplied to the processor one by one (ACCESS TIME).

Design Issue of Memory System

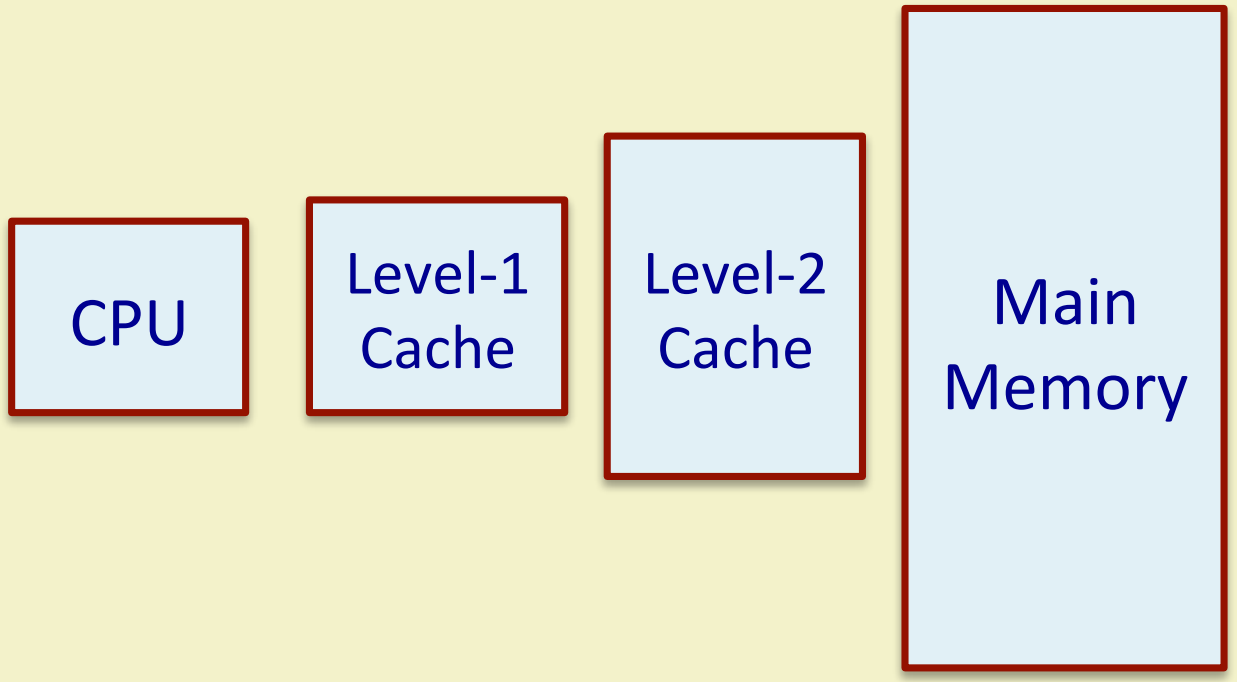
- The most important issue is to bridge the processor-memory gap that has been widening with every passing year.
  - Advancements in memory technology are unable to cope with faster advancements in processor technology.



- Some important questions?
  - How to make the memory system work faster?
  - How to increase the data transfer rate between CPU and memory?
  - How to address the ever increasing storage needs of applications?
- Some possible solutions:
  - *Cache Memory*: to increase the effective speed of the memory system.
  - *Virtual Memory*: to increase the effective size of the memory system.

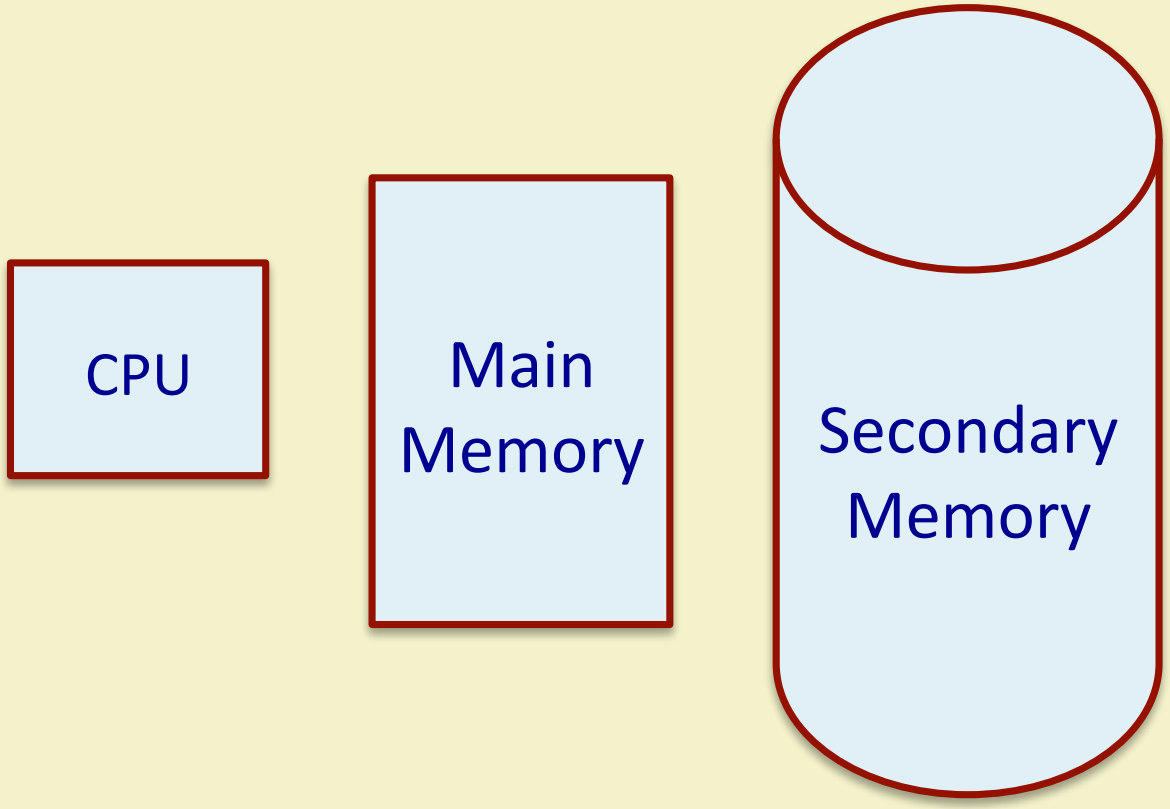
What is Cache Memory?

- A fast memory (possibly organized in several levels) that sits between processor and main memory.
- Faster than main memory and relatively small.
- Frequently accessed data and instructions are stored here.
- Cache memory makes use of the fast SRAM technology.



What is Virtual Memory?

- Technique used by the operating system to provide an illusion of very large memory to the processor.
- Program and data are actually stored on secondary memory that is much larger.
- Transfer parts of program and data from secondary memory to main memory only when needed.



Introduction

- Broadly two types of semiconductor memory systems:
  a) Static Random Access Memory (SRAM)
  b) Dynamic Random Access Memory (DRAM)
     i. Asynchronous DRAM
     ii. Synchronous DRAM
- Vary in terms of speed, density, volatility properties, and cost.
  – Present-day main memory systems are built using DRAM.
  – Cache memory systems are built using SRAM.

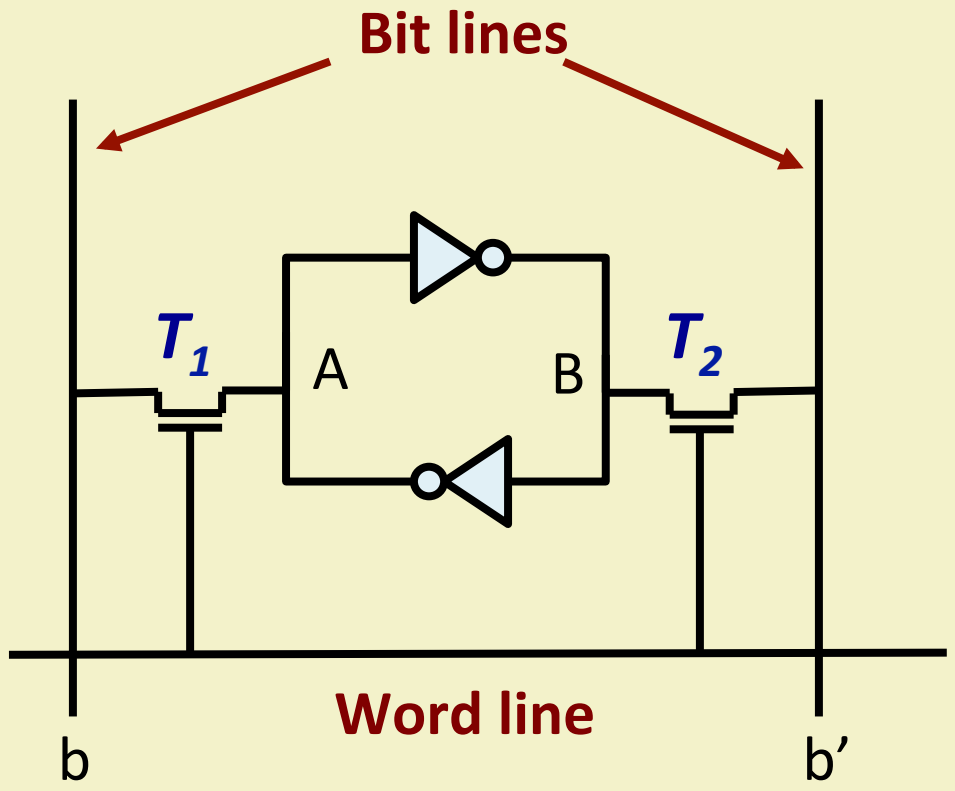
Static Random Access Memory (SRAM)

- SRAM consists of circuits which can store the data as long as power is applied.
- It is a type of semiconductor memory that uses bistable latching circuitry (flip-flop) to store each bit.
- SRAM memory arrays can be arranged in rows and columns of memory cells.
  - Called *word line* and *bit line*.

- SRAM technology:
  - Can be built using 4 or 6 MOS transistors.
  - Modern SRAM chips in the market uses 6-transistor implementations for CMOS compatability.
  - Widely used in small-scale systems like microcontrollers and embedded systems.
  - Also used to implement cache memories in computer systems.
    - To be discussed later.

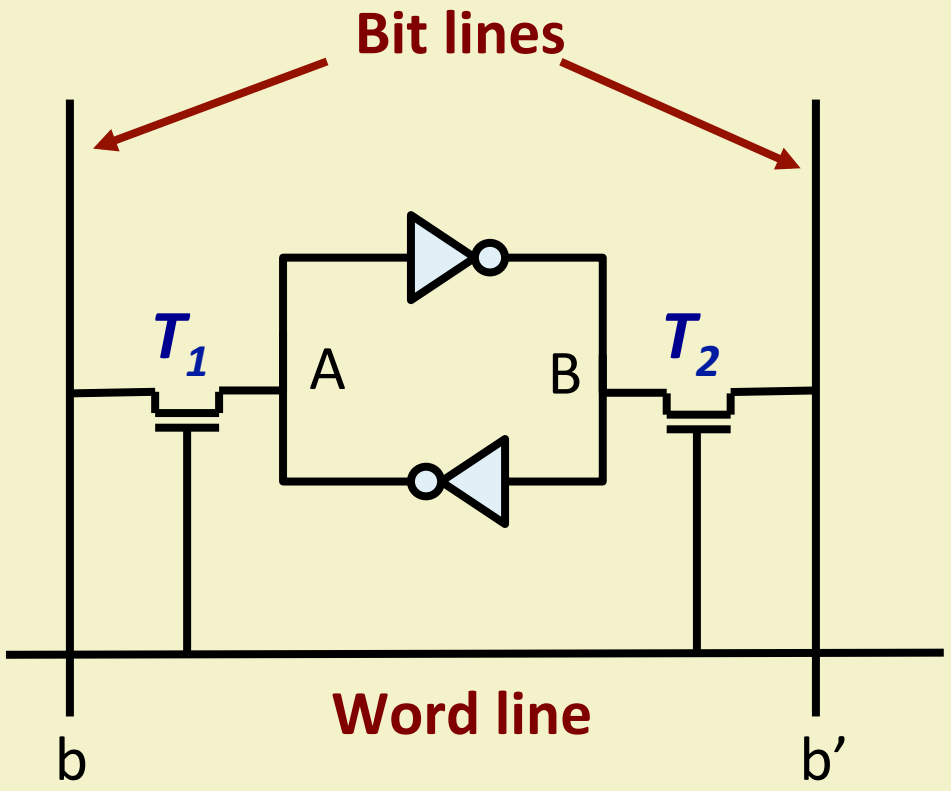
A 1-bit SRAM Cell

- Two inverters are cross connected to form a latch.
- The latch is connected to two bit lines with transistors *T1* and *T2*.
- Transistors behave like switches that can be opened (OFF) or closed (ON) under the control of the word line.
- To retain the state of the latch, the word line can be grounded which makes the transistors off.



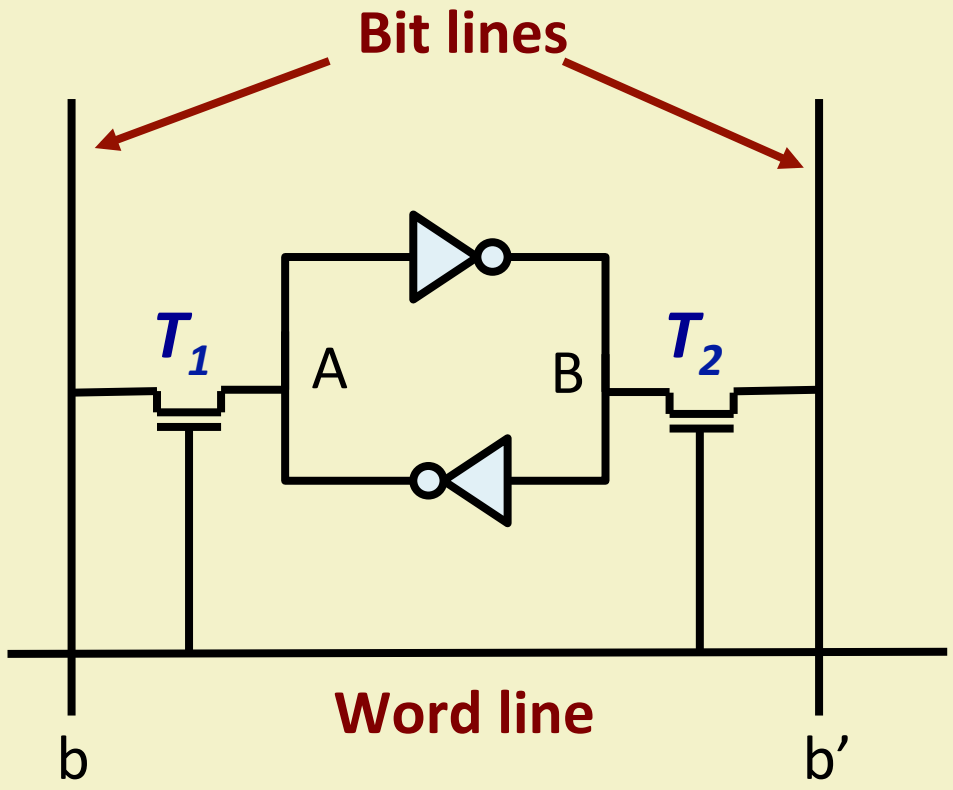
READ Operation in SRAM

- To read the content of the cell, the word line is activated (= *1*) to make the transistors *T1* and *T2* on.
- The value stored in latch is available on bit line *b* and its complement on *b'*.
- Sense/write circuits connected to the bit lines monitor the states of *b* and *b'*.



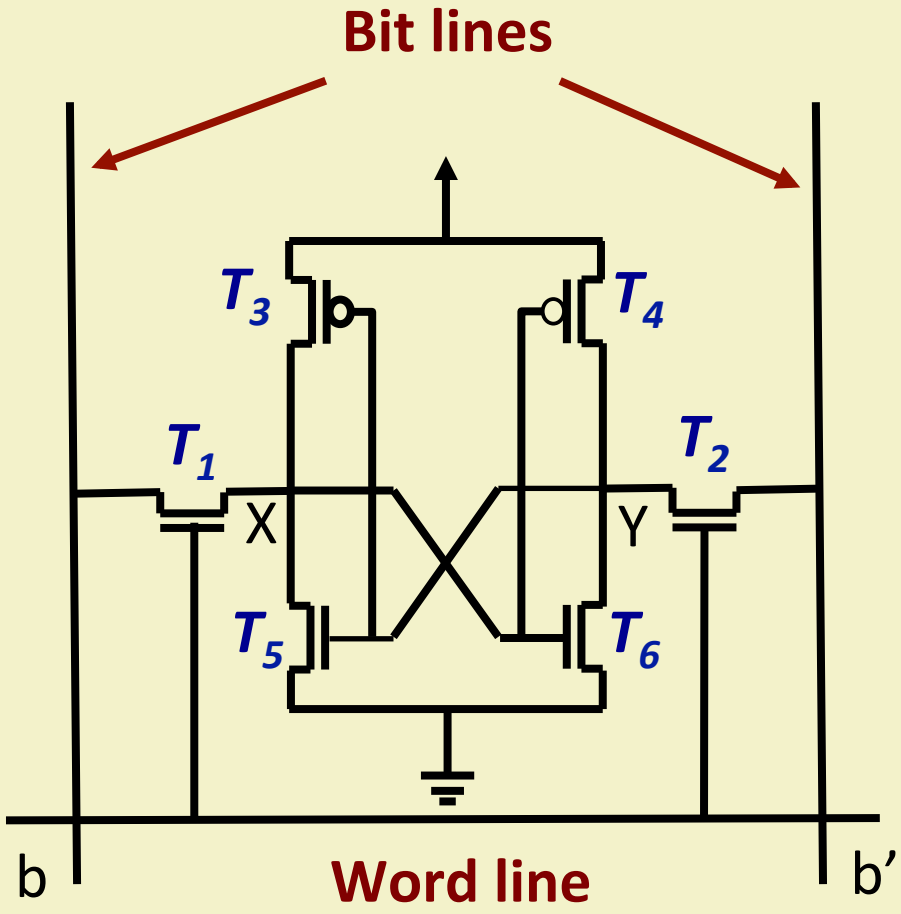
WRITE Operation in SRAM

- **To write 1**: The bit line *b* is set with *1* and bit line *b'* is set with *0*. Then the word line is activated and the data is written to the latch.
- **To write 0**: The bit line *b* is set with *0* and bit line *b'* is set with *1*. Then the word line is activated and the data is written to the latch.
- The required signals (either *1* or *0*) are generated by the sense/write circuit.



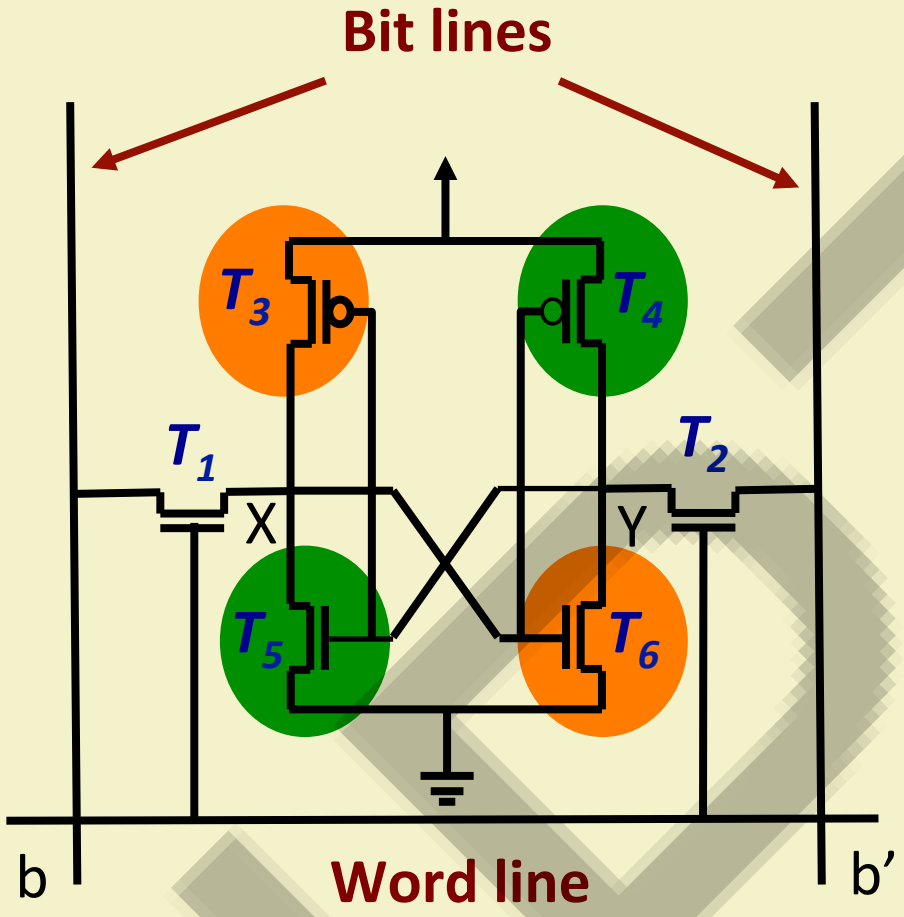
6-Transistor Static Memory cell

- 1-bit SRAM cell with 6-transistors are used in modern-day SRAM implementations.
- Transistors (*T3* &*T5*) and (*T4* &*T6*) form the CMOS inverters in the latch.
- The data can be read or written in the same way as explained.



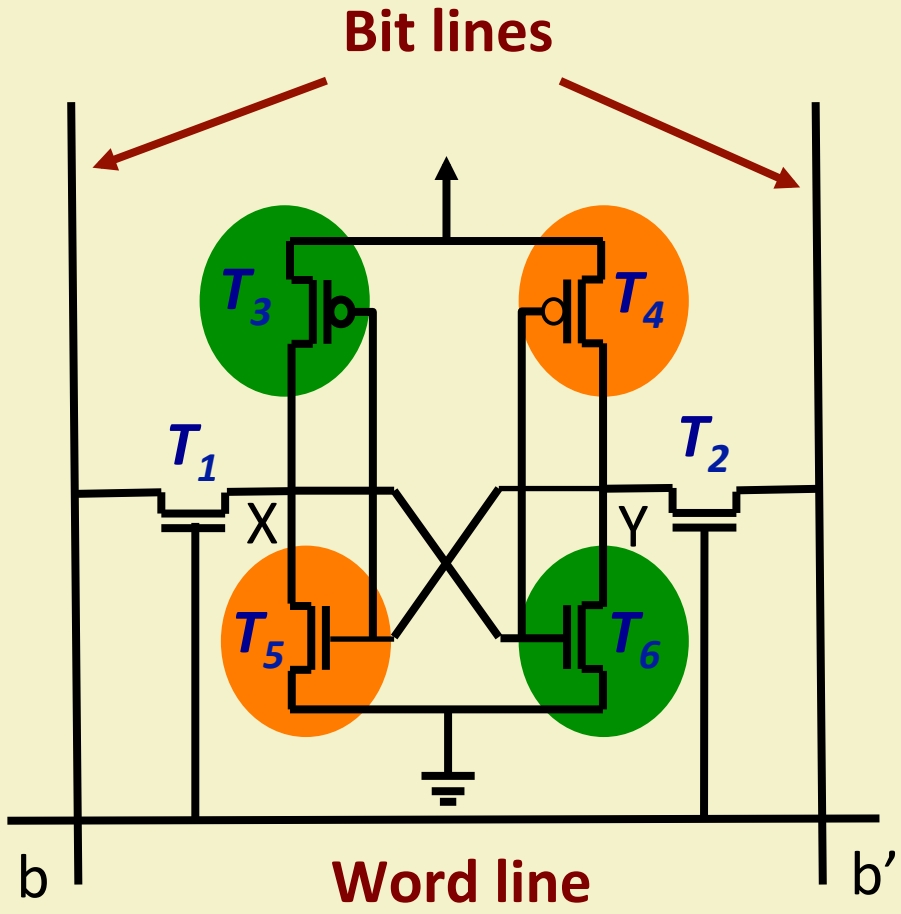
In State 0

- In state 0 the voltage at *X* is low and the voltage at *Y* is high.
- When the voltage at *X* is low, transistors (*T4* & *T5*) are on while (*T3* & *T6*) are off.
- When word line is activated, *T1* and *T2* are turned on and the bit lines *b* will have *0* and *b'* will have *1*.



In State 1

- In state 1 the voltage at *X* is high and the voltage at *Y* is low.
- When the voltage at *X* is high, transistors (*T3* & *T6*) are on while (*T4* & *T5*) are off.
- When word line is activated, *T1* and *T2* are turned on and the bit lines *b* will have *1* and *b'* will have *0*.

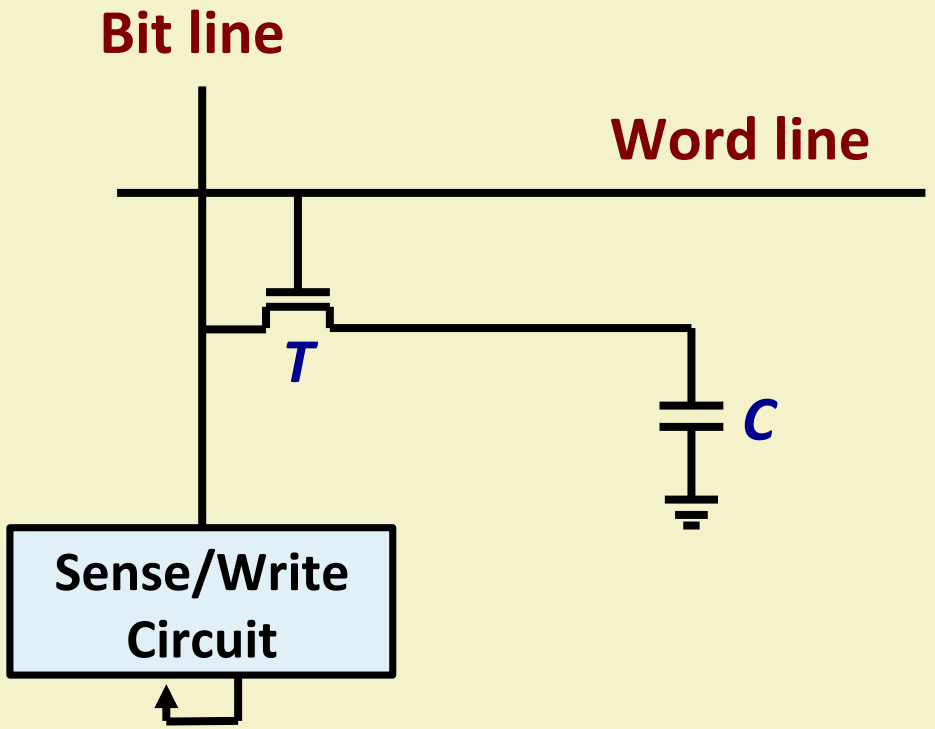


Features of SRAM

- Moderate / High power consumption.
  - Current flows in the cells only when the cell is accessed.
  - Because of latch operation, power consumption is higher than DRAM.
- Simplicity – refresh circuitry is not needed.
  - Volatile :: continuous power supply is required.
- Fast operation.
  - Access time is very fast; fast memories (cache) are built using SRAM.
- High cost.
  - 6 transistors per cell.
- Limited capacity.
  - Not economical to manufacture high-capacity SRAM chips.

Dynamic Random Access Memory (DRAM)

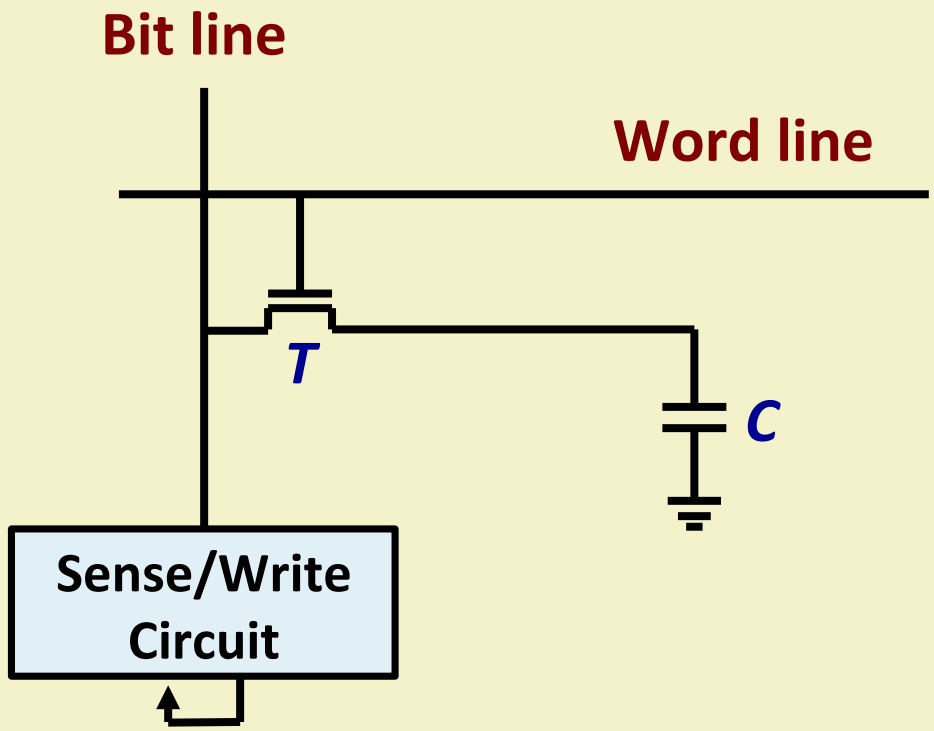
- Dynamic RAM do not retain its state even if power supply is on.
  - Data stored in the form of charge stored on a capacitor.
- Requires periodic refresh.
  - The charge stored cannot be retained over long time (due to leakage).
- Less expensive that SRAM.
  - Requires less hardware (one transistor and one capacitor per cell).
- Address lines are multiplexed.



1-transistor DRAM Cell

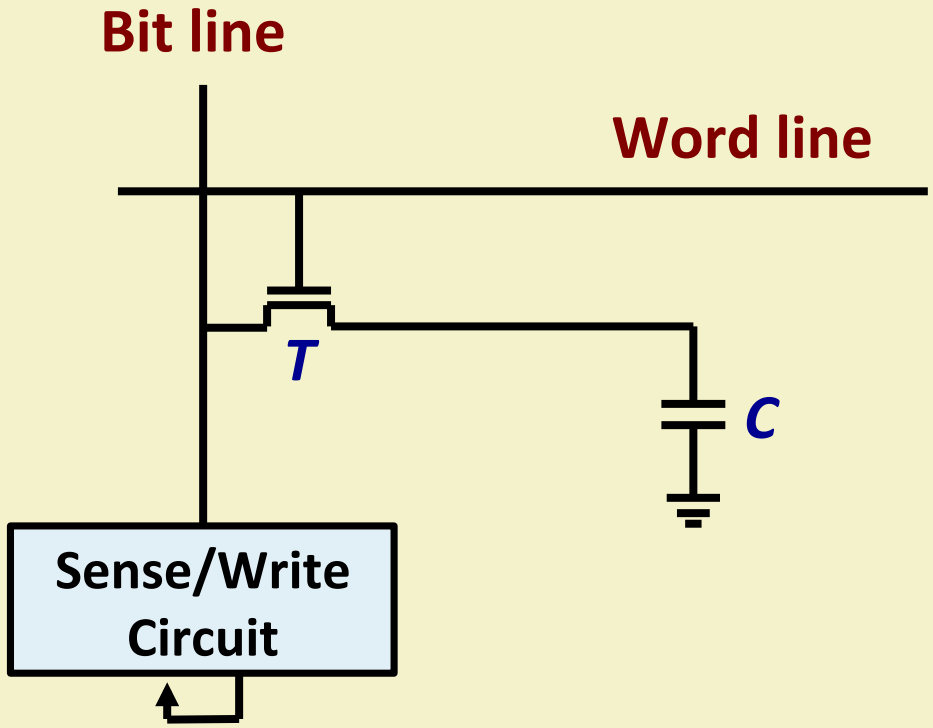
READ Operation in DRAM

- The transistor of the particular cell is turned on by activating the word line.
- A sense amplifier connected to bit line senses the charge stored in the capacitor.
- If the charge is above threshold, the bit line is maintained at high voltage, which represents logic *1*.
- If the charge is below threshold, the bit line is grounded, which represent logic *0*.



WRITE Operation in DRAM

- The transistor of the particular cell is turned on by activating the word line.
- Depending on the value to be written (*0* or *1*), an appropriate voltage is applied to the bit line.
- The capacitor gets charged to the required voltage state.
- Refreshing of the capacitor requires periodic READ-WRITE cycles (every few msec).



Introduction

- Programmers want unlimited amount of memory with very low latency.
- Fast memory technology is more expensive per bit than slower memory.
  - SRAM is more expensive than DRAM, DRAM is more expensive than disk.
- Possible solution?
  - Organize the memory system in several levels, called *memory hierarchy*.
  - Exploit temporal and spatial locality on computer programs.
  - Try to keep the commonly accessed segments of program / data in the faster memories.
  - Results in faster access times on the average.

Quick Review of Memory Technology

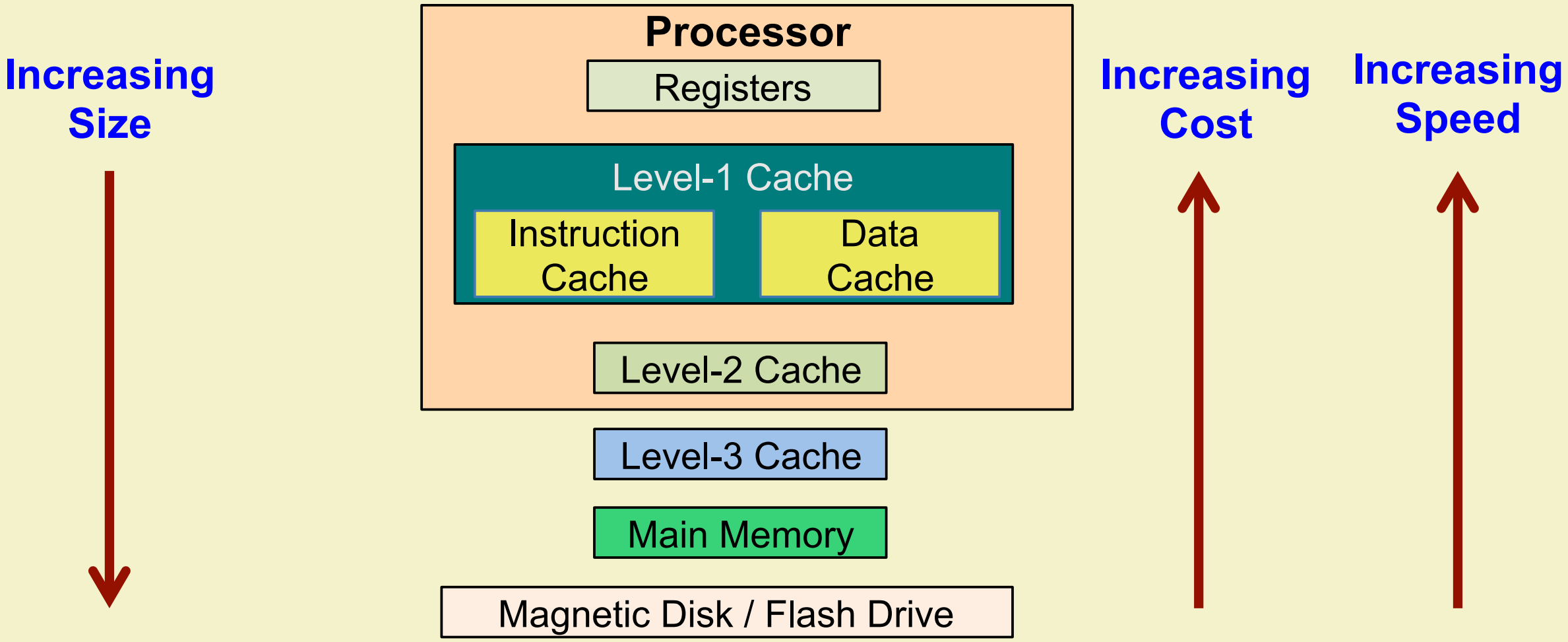
- Static RAM:
  - Very fast but expensive memory technology (requires 6 transistors / bit).
  - Packing density is limited.
- Dynamic RAM:
  - Significantly slower than DRAM, but much less expensive (1 transistor / bit).
  - Requires periodic refreshing.
- Flash memory:
  - Non-volatile memory technology that uses floating-gate MOS transistors.
  - Slower than DRAM, but higher packing density, and lower cost per bit.

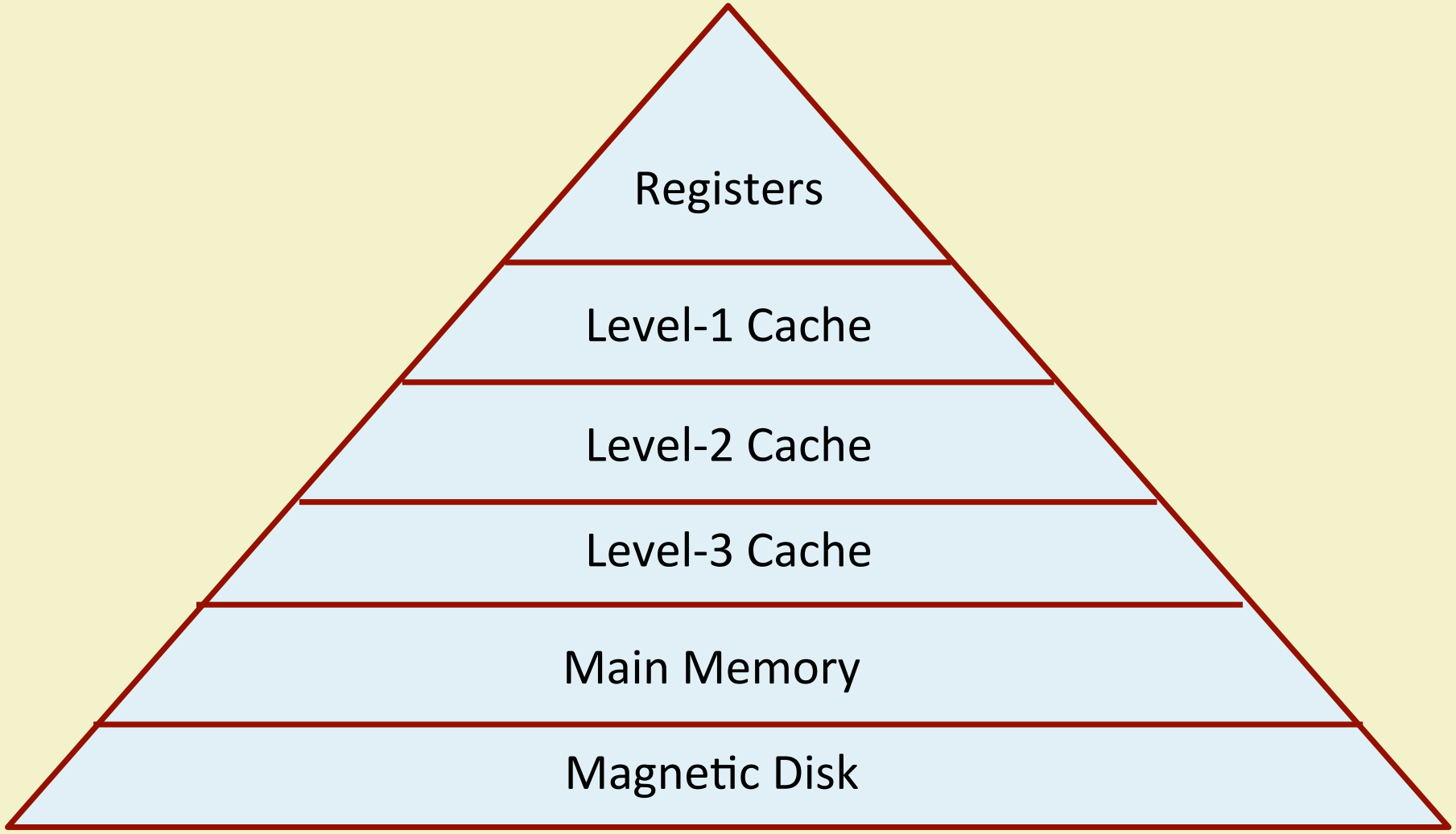
- Magnetic disk:
  - Provides large amount of storage, with very low cost per bit.
  - Much slower than DRAM, and also flash memory.
  - Requires mechanical moving parts, and uses magnetic recording technology.

Memory Hierarchy

- The memory system is organized in several levels, using progressively faster technologies as we move towards the processor.
  - The entire addressable memory space is available in the largest (but slowest) memory (typically, magnetic disk or flash storage).
  - We incrementally add smaller (but faster) memories, each containing a subset of the data stored in the memory below it.
  - We proceed in steps towards the processor.

- Typical hierarchy (starting with closest to the processor):
  1. Processor registers
  2. Level-1 cache (typically divided into separate instruction and data cache)
  3. Level-2 cache
  4. Level-3 cache
  5. Main memory
  6. Secondary memory (magnetic disk / flash drive)
- As we move away from the processor:
  - Size increases
  - Cost decreases
  - Speed decreases


Increasing Size
Processor
Registers
Level-1 Cache
Instruction Cache
Data Cache
Level-2 Cache
Level-3 Cache
Main Memory
Magnetic Disk / Flash Drive
Increasing Cost
Increasing Speed

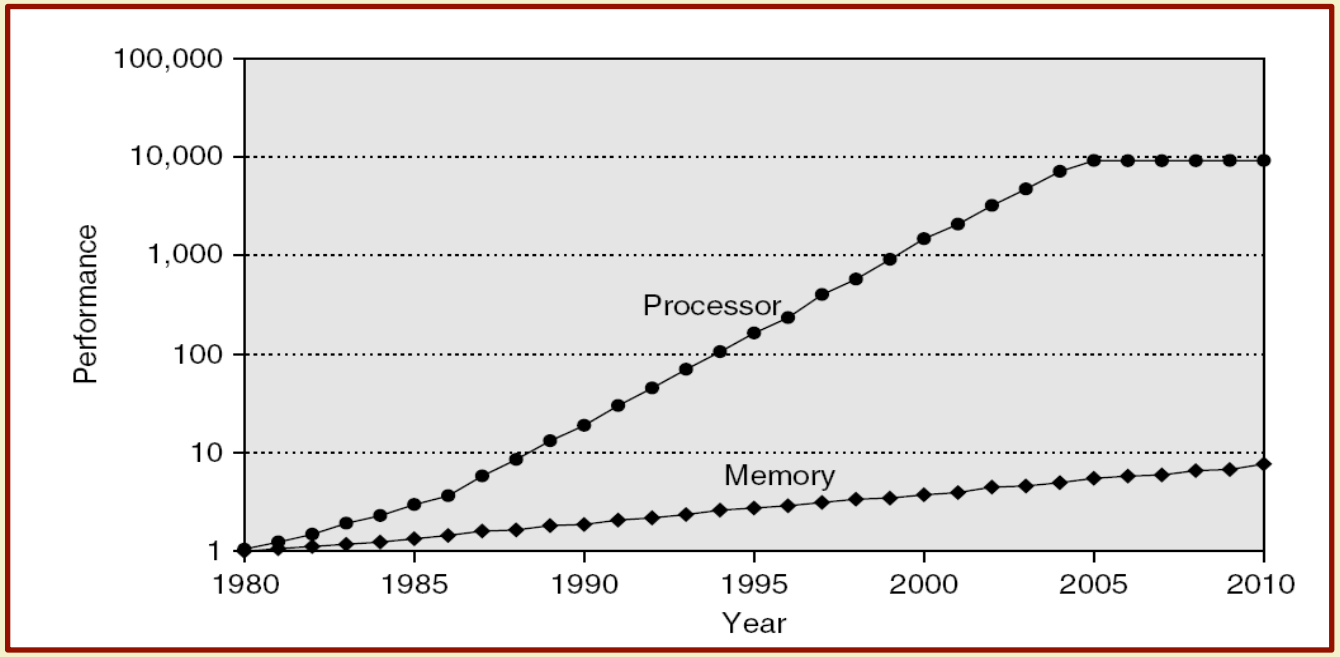

Registers
Level-1 Cache
Level-2 Cache
Level-3 Cache
Main Memory
Magnetic Disk

A Comparison

Level	Typical Access Time	Typical Capacity	Other Features
Register	300-500 ps	500-1000 B	On-chip
Level-1 cache	1-2 ns	16-64 KB	On-chip
Level-2 cache	5-20 ns	256 KB – 2 MB	On-chip
Level-3 cache	20-50 ns	1-32 MB	On or off chip
Main memory	50-100 ns	1-16 GB	
Magnetic disk	5-50 ms	100 GB – 16 TB	

Major Obstacle in Memory System Design

- Processor is much faster than main memory.
  - Has to spend much of the time waiting while instructions and data are being fetched from main memory.
  - Memory speed cannot be increased beyond a certain point.



Impact of Processor / Memory Performance Gap

Year	CPU Clock	Clock Cycle	Memory Access	Minimum CPU Stall Cycles
1986	8 MHz	125 ns	190 ns	190 / 125 – 1 = 0.5
1989	33 MHz	30 ns	165 ns	165 / 30 – 1 = 4.5
1992	60 MHz	16.6 ns	120 ns	120 / 16.6 – 1 = 6.2
1996	200 MHz	5 ns	110 ns	110 / 5 – 1 = 21.0
1998	300 MHz	3.33 ns	100 ns	100 / 3.33 – 1 = 29.0
2000	1 GHz	1 ns	90 ns	90 / 1 – 1 = 89.0
2002	2 GHz	0.5 ns	80 ns	80 / 0.5 – 1 = 159.0
2004	3 GHz	0.33 ns	60 ns	60 / 0.33 – 1 = 179.0

Ideal memory access time = 1 CPU cycle

Real memory access time >> 1 CPU cycle

- Memory Latency Reduction Techniques:
  - Faster DRAM cells (depends on VLSI technology)
  - Wider memory bus width (fewer memory accesses needed)
  - Multiple memory banks
  - Integration of memory controller with processor
  - New emerging RAM technologies
- Memory Latency Hiding Techniques
  - Memory hierarchy (using SRAM-based cache memories)
  - Pre-fetching instructions and/or data from memory before they are actually needed (used to hide long memory access latency)

Locality of Reference

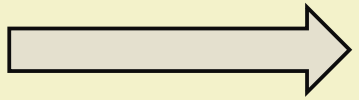
- Programs tend to reuse data and instructions they have used recently.
  - Rule of thumb: 90% of the total execution time of a program is spent in only 10% of the code (also called 90/10 rule).
  - Reason: nested loops in a program, few procedures calling each other repeatedly, arrays of data items being accessed sequentially, etc.
- Basic idea to exploit this rule:
  - Based on a program's recent past, we can predict with a reasonable accuracy what instructions and data will be accessed in the near future.

- The 90/10 rule has two dimensions:
  a) Temporal Locality (locality in time)
     - If an item is referenced in memory, it will tend to be referenced again soon.
  b) Spatial locality (locality in space)
     - If an item is referenced in memory, nearby items will tend to be referenced soon.

(a) Temporal Locality

- Recently executed instructions are likely to be executed again very soon.
- Example: computing factorial of a number.

```
fact = 1;
for  k = 1 to N
  fact = fact * k;
```



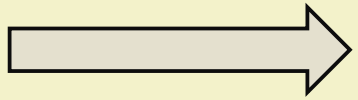
```
        ADDI    $t1,$zero,1
        ADDI    $t2,$zero,N
        ADDI    $t3,$zero,1
Loop:   MUL     $t1,$t1,$t3
        ADDI    $t3,$t3,1
        SGT     $t4,$t3,$t2
        BNEZ    $t4,Loop
```

- The four instructions in the loop are executed more frequently than the others.

(b) Spatial Locality

- Instructions residing close to a recently executing instruction are likely to be executed soon.
- Example: accessing elements of an array.

```
sum = 0;
for  k = 1 to N
  sum = sum + A[k];
```

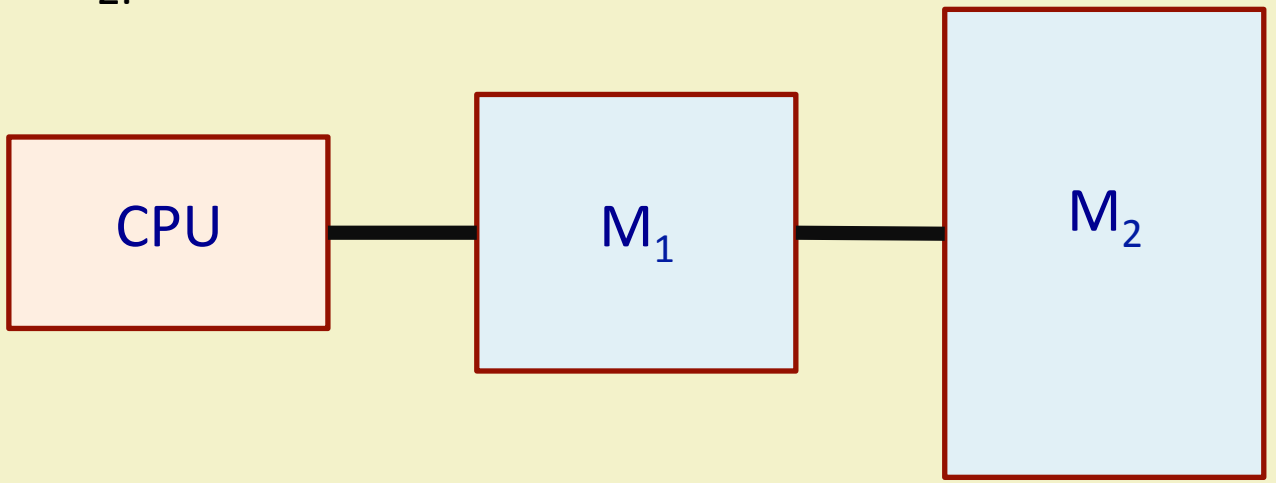


```
        SUB     $t1,$t1,$t1
        ADDI    $t2,$zero,N
        ADDI    $t3,$zero,1
        ADDI    $t5,$zero,A
Loop:   LW      $t8,0($t5)
        ADD     $t1,$t1,$t8
        ADDI    $t3,$t3,1
        SGT     $t4,$t3,$t2
        BNEZ    $t4,Loop
```

- Performance can be improved by copying the array into cache memory.

Performance of Memory Hierarchy

- We first consider a 2-level hierarchy consisting of two levels of memory, say, M_1 and M_2.



- Cost:
  - Let c_i denote the cost per bit of memory M_i, and S_i denote the storage capacity in bits of M_i.
  - The average cost per bit of the memory hierarchy is given by:

$$\text{Cost } c = \frac{c_1 S_1 + c_2 S_2}{S_1 + S_2}$$

  - In order to have $c \rightarrow c_2$, we must ensure that $S_1 << S_2$.

- Hit Ratio / Hit Rate:
  - The hit ratio H is defined as the probability that a logical address generated by the CPU refers to information stored in M_1.
  - We can determine H experimentally as follows:
    - A set of representative programs is executed or simulated.
    - The number of references to M_1 and M_2, denoted by N_1 and N_2 respectively, are recorded.

$$H = \frac{N_1}{N_1 + N_2}$$

  - The quantity (1 – H) is called the miss ratio.

- Access Time:
  - Let t_{A1} and t_{A2} denote the access times of M_1 and M_2 respectively, relative to the CPU.
  - The average time required by the CPU to access a word in memory can be expressed as:

$$t_A = H.t_{A1} + (1 - H).t_{MISS}$$

where t_{MISS} denotes the time required to handle the miss, called miss penalty.

- The miss penalty t_{MISS} can be estimated in various ways:
  a) The simplest approach is to set $T_{MISS} = t_{A2}$, that is, when there is a miss the data is accessed directly from M_2.
  b) A request for a word not in M_1 typically causes a block containing the requested word to be transferred from M_2 to M_1. After completion of the block transfer, the word can be accessed in M_1.

  If t_B denotes the block transfer time, we can write

  $$t_{MISS} = t_B + t_{A1} \qquad [\text{since } t_B >> t_{A1} ,\ t_{A2} \approx t_B]$$

  Thus, $t_A = H.t_{A1} + (1 – H).(t_B + t_{A1})$

  c) If t_{HIT} denotes the time required to check whether there is a hit, we can write

  $$t_{MISS} = t_{HIT} + t_B + t_{A1}$$

- Efficiency:
  - Let $r = t_{A2} / t_{A1}$ denote the access time ratio of the two levels of memory.
  - We define the access efficiency as $e = t_{A1} / t_A$, which is the factor by which t_A differs from its minimum possible value.

$$\text{Efficiency } e = \frac{t_{A1}}{H.t_{A1} + (1 - H).t_{A2}} = \frac{1}{H + (1 - H).r}$$

- Speedup:
  - The speedup gained by using the memory hierarchy is defined as $S = t_{A2} / t_A$.
  - We can write:

$$S = \frac{t_{A2}}{H.t_{A1} + (1 - H).t_{A2}} = \frac{1}{H / r + (1 - H)}$$

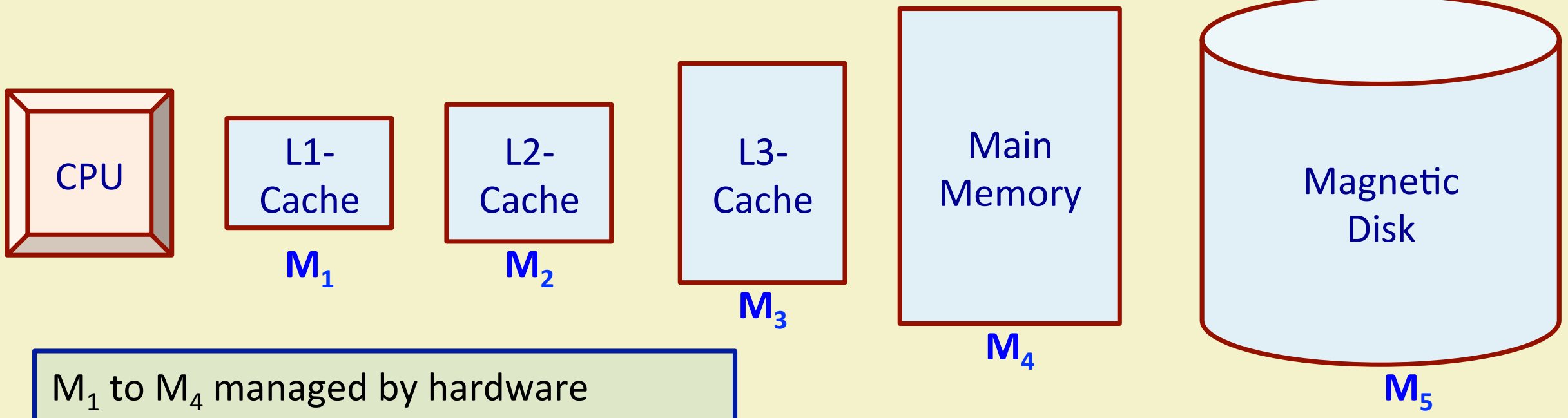
  - The same result follows from Amadahl's law.

Some Common Terminologies Used

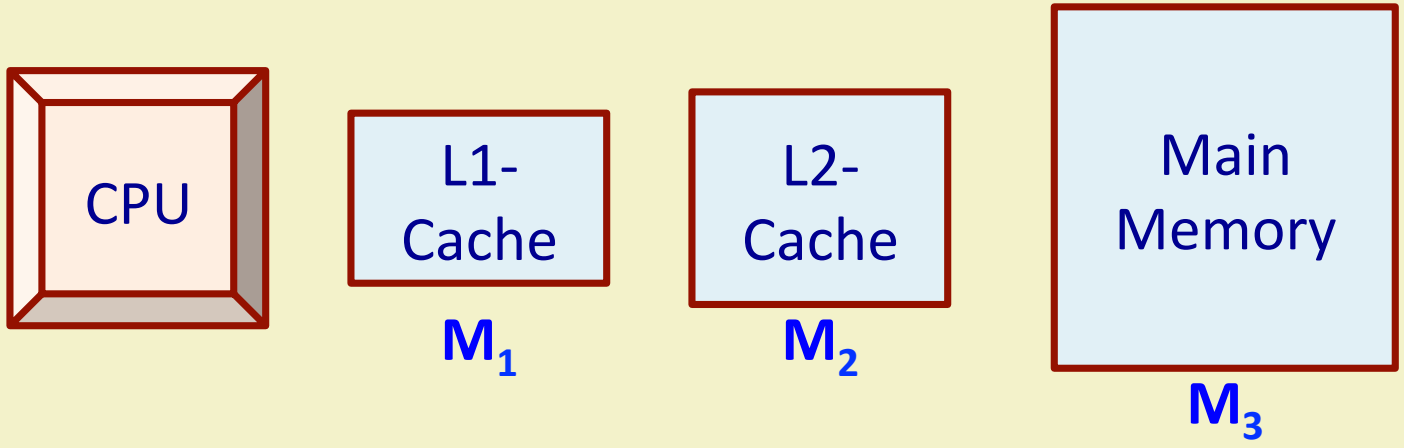
- Block: The smallest unit of information transferred between two levels.
- Hit Rate: The fraction of memory accesses found in the upper level.
- Hit Time: Time to access the upper level
  - Upper level access time + Time to determine hit/miss
- Miss: Data item needs to be retrieved from a block in the lower level.
- Miss Rate: The fraction of memory accesses not found in the upper level.
- Miss Penalty: Overhead whenever a miss occurs.
  - Time to replace a block in the upper level + Time to transfer the missed block

Performance Calculation for Multi-Level Hierarchy

- Most of the practical memory systems use more than 2 levels of hierarchy.



M_1 to M_4 managed by hardware
M_4 to M_5 managed by operating system



t_{L1} : access time of M_1

t_{L2} : access time of M_2

H_{L1} : hit ratio of M_1

H_{L2} : hit ratio of M_2 with respect to the residual accesses that try to access M_2

- Consider a 3-level hierarchy consisting of L1-cache, L2-cache and main memory.
- Whenever there is a miss in L1, we go to L2.
- Average access time can be calculated as:

$$t_A = H_{L1}.t_{L1} + (1 - H_{L1}) . [H_{L2}.t_{L2} + (1 - H_{L2}).t_{MISS2}]$$

- Here, t_{MISS2} is the miss penalty when the requested data is found neither in M_1 nor in M_2.

Implications of a Memory Hierarchy to the CPU

- Processors designed without memory hierarchy are simpler because all memory accesses take the same amount of time.
  - Misses in a memory hierarchy implies variable memory access times as seen by the CPU.
- Some mechanism is required to determine whether or not the requested information is present in the top level of the memory hierarchy.
  - Check happens on every memory access and affects hit time.
  - Implemented in hardware to provide acceptable performance.

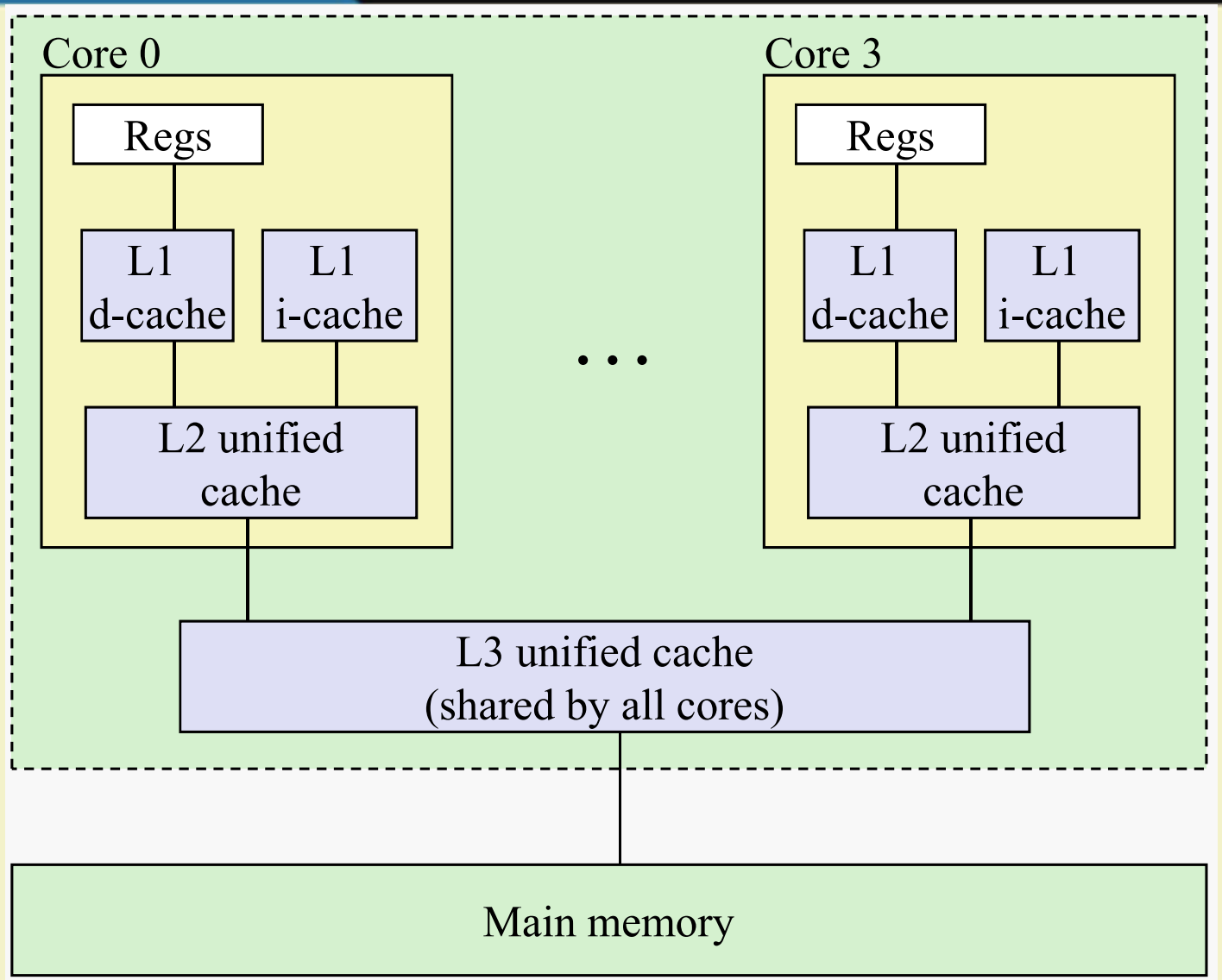
- Some mechanism is required to transfer blocks between consecutive levels.
  - If the block transfer requires 10's of clock cycles (like in cache / main memory hierarchy), it is controlled by hardware.
  - If the block transfer requires 1000's of clock cycles (like in main memory / secondary memory hierarchy), it can be controlled by software.
- Four main questions:
  1. *Block Placement*: Where to place a block in the upper level?
  2. *Block Identification*: How is a block found if present in the upper level?
  3. *Block Replacement*: Which block is to be replaced on a miss?
  4. *Write Strategy*: What happens on a write?

Common Memory Hierarchies

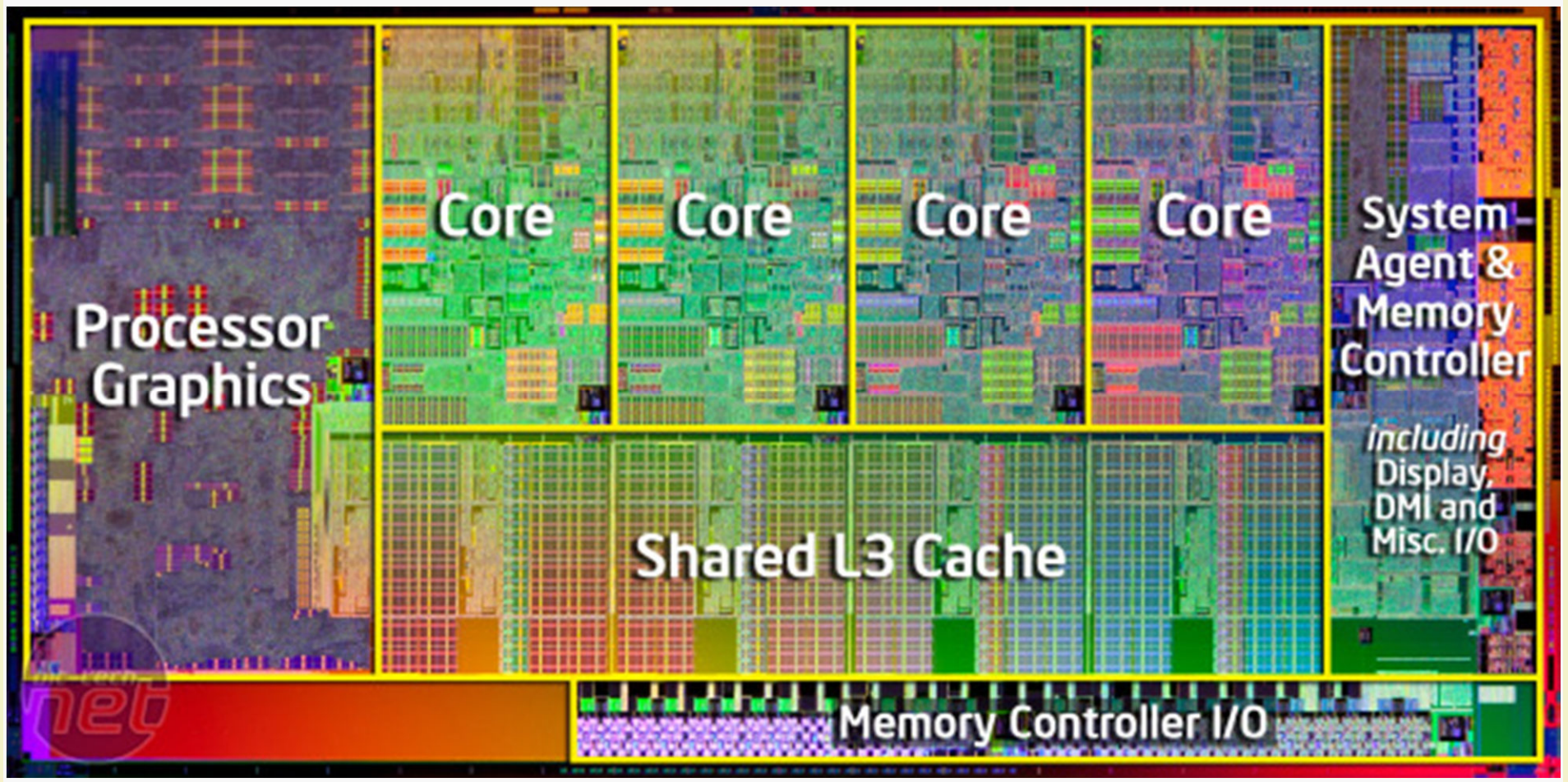
- In a typical computer system, the memory system is managed as two different hierarchies.
  - The Cache / Main Memory hierarchy, which consists of 2 to 4 levels and is managed by hardware.
    - Main objective: provide fast average memory access.
  - The Main Memory / Secondary Memory hierarchy, which consists of 2 levels and is managed by software (operating system).
    - Main objective: provide large memory space for users (virtual memory).

Intel Core-i7 Cache Hierarchy

- L1 i-cache and d-cache:
  - 32 KB, 8-way set associative
  - Access: 4 cycles
- L2 unified cache:
  - 256 KB, 8-way set associative
  - Access: 11 cycles
- L3 unified cache:
  - 8 MB, 16-way set associative
  - Access: 30-40 cycles
- Block size: 64 bytes for all caches



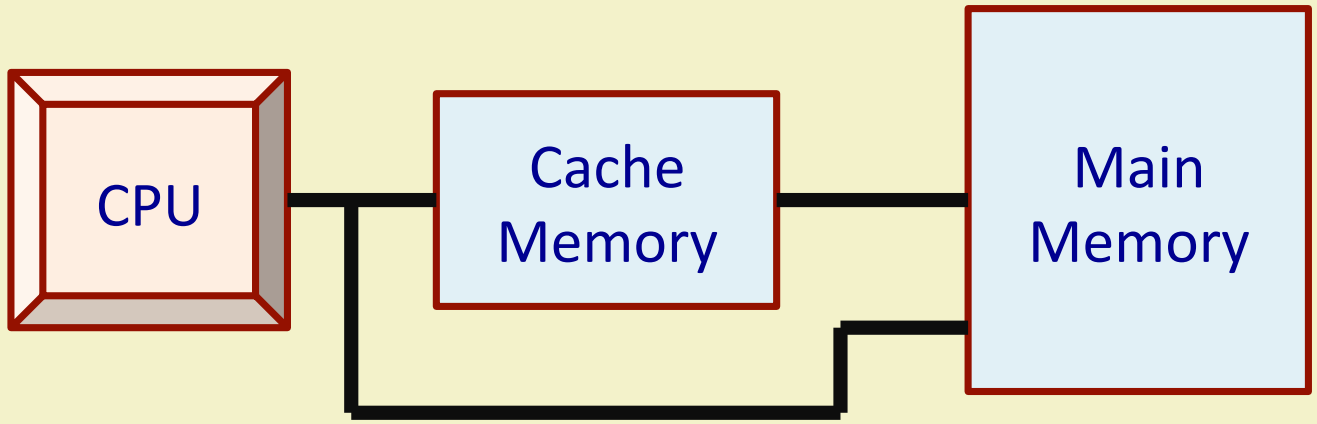
Core-i7 Sandybridge



- Every core has its own L1 and L2 caches.
- The L3 cache is shared by all the cores and is also on-chip.

Introduction

- Let us consider a single-level cache, and that part of the memory hierarchy consisting of cache memory and main memory.



- Cache memory is logically divided into *blocks* or *lines*, where every block (line) typically contains 8 to 256 bytes.
- When the CPU wants to access a word in memory, a special hardware first checks whether it is present in cache memory.
  - If so (called *cache hit*), the word is directly accessed from the cache memory.
  - If not, the block containing the requested word is brought from main memory to cache.
  - For writes, sometimes the CPU can also directly write to main memory.
- Objective is to keep the commonly used blocks in the cache memory.
  - Will result in significantly improved performance due to the property of locality of reference.

Q1. Where can a block be placed in the cache?

- This is determined by some mapping algorithms.
  - Specifies which main memory blocks can reside in which cache memory blocks.
  - At any given time, only a small subset of the main memory blocks can be held in main memory.
- Three common block mapping techniques are used:
  a) Direct Mapping
  b) Associative Mapping
  c) (N-way) Set Associative Mapping
- The algorithms shall be explained with the help of an example.

Example: A 2-level memory hierarchy

- Consider a 2-level cache memory / main memory hierarchy.
  - The cache memory consists of 256 blocks (lines) of 32 words each.
    - Total cache size is 8192 (8K) words.
  - Main memory is addressable by a 24-bit address.
    - Total size of the main memory is 2^{24} = 16 M words.
    - Number of 32-word blocks in main memory = 16 M / 32 = 512K

(a) Direct Mapping

- Each main memory block can be placed in only one block in the cache.
- The mapping function is:

  Cache Block = (Main Memory Block) % (Number of cache blocks)

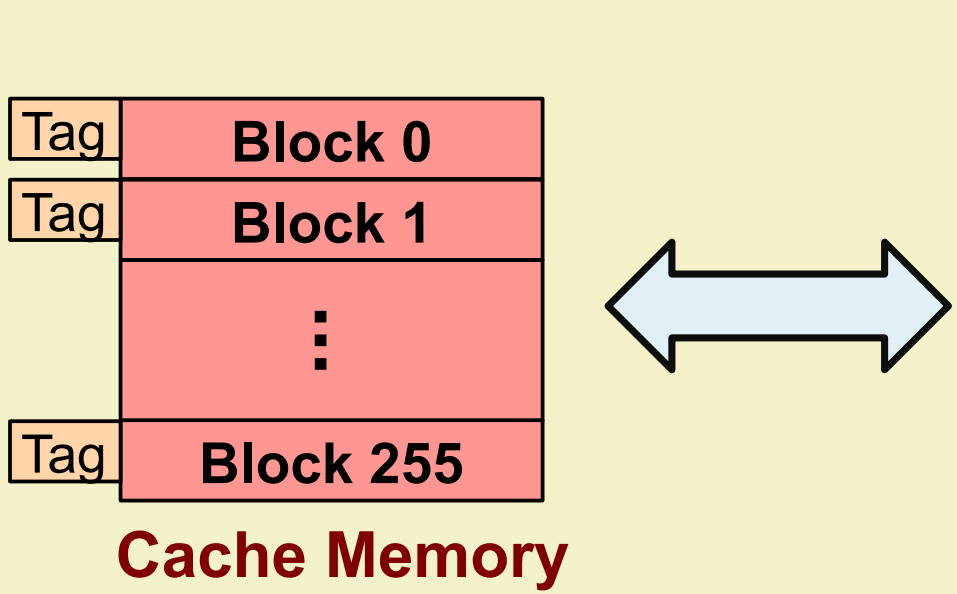
- For the example,

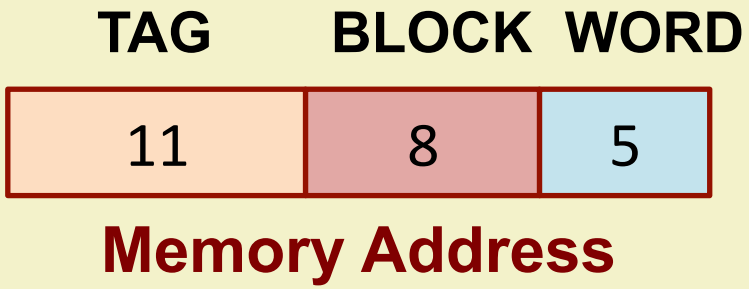
  Cache Block = (Main Memory Block) % 256

- Some example mappings:

  0 → 0, 1 → 1, 255 → 255, 256 → 0, 257 → 1, 512 → 0, 512 → 1, etc.

Direct Mapping



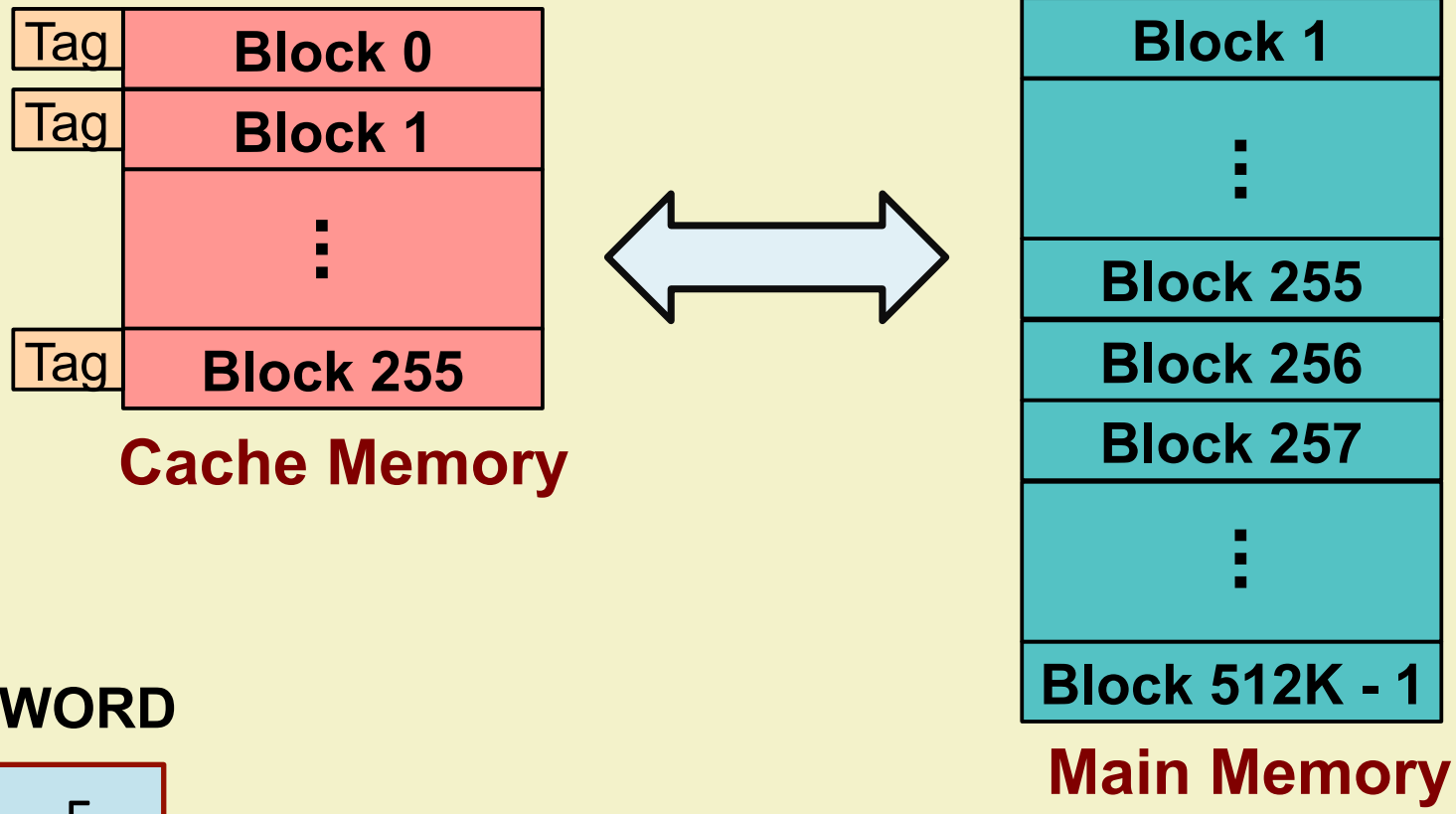


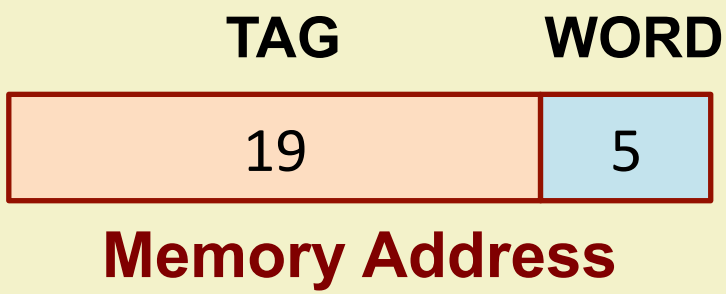
- Block replacement algorithm is trivial, as there is no choice.
- More than one MM block is mapped onto the same cache block.
  - May lead to contention even if the cache is not full.
  - New block will replace the old block.
  - May lead to poor performance if both the blocks are frequently used.
- The MM address is divided into three fields: TAG, BLOCK and WORD.
  - When a new block is loaded into the cache, the 8-bit BLOCK field determines the cache block where it is to be stored.
  - The high-order 11 bits are stored in a TAG register associated with the cache block.
  - When accessing a memory word, the corresponding TAG fields are compared.
    - Match implies HIT.

(b) Associative Mapping

- Here, a MM block can potentially reside in any cache block position.
- The memory address is divided into two fields: TAG and WORD.
  - When a block is loaded into the cache from MM, the higher order 19 bits of the address are stored into the TAG register corresponding to the cache block.
  - When accessing memory, the 19-bit TAG field of the address is compared with *all the TAG registers* corresponding to all the cache blocks.
- Requires associative memory for storing the TAG values.
  - High cost / lack of scalability.
- Because of complete freedom in block positioning, a wide range of replacement algorithms is possible.

Fully Associative Mapping



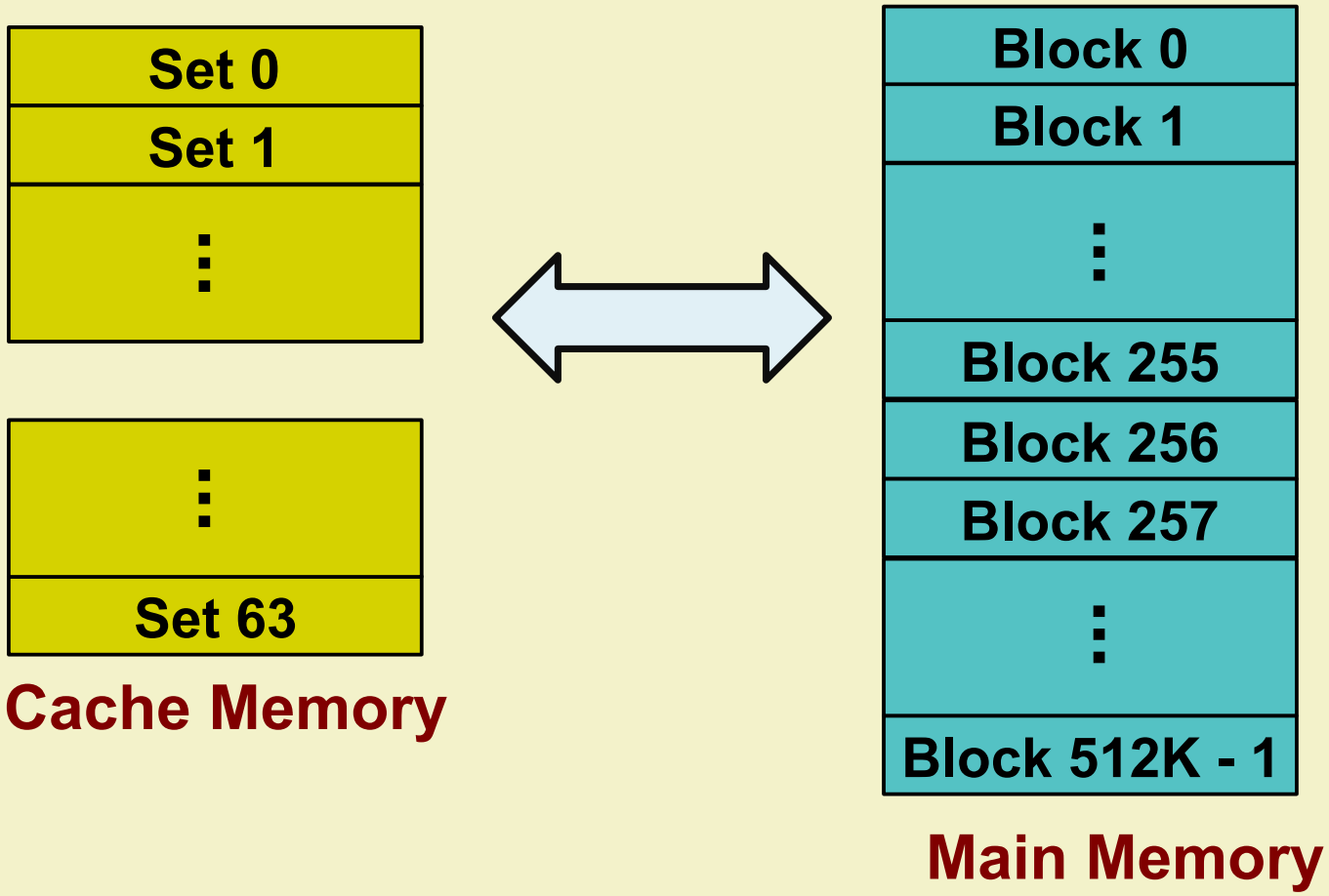


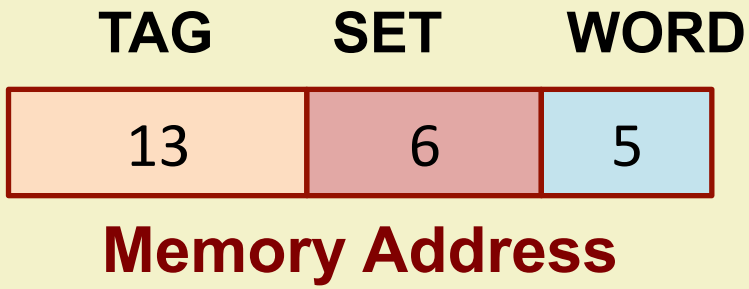
(c) N-way Set Associative Mapping

- A group of N consecutive blocks in the cache is called a set.
- This algorithm is a balance of direct mapping and associative mapping.
  - Like direct mapping, a MM block is mapped to a set.

    Set Number = (MM Block Number) % (Number of Sets in Cache)
  - The block can be placed anywhere within the set (there are N choices)
- The value of N is a design parameter:
  - N = 1 :: same as direct mapping.
  - N = number of cache blocks :: same as associative mapping.
  - Typical values of N used in practice are: 2, 4 or 8.

4-way Set Associative Mapping





- Illustration for N = 4:
  - Number of sets in cache memory = 64.
  - Memory blocks are mapped to a set using modulo-64 operation.
  - Example: MM blocks 0, 64, 128, etc. all map to set 0, where they can occupy any of the four available positions.
- MM address is divided into three fields: TAG, SET and WORD.
  - The TAG field of the address must be associatively compared to the TAG fields of the 4 blocks of the selected set.
  - This instead of requiring a single large associative memory, we need a number of very small associative memories only one of which will be used at a time.

Q2. How is a block found if present in cache?

- Caches include a TAG associated with each cache block.
  - The TAG of every cache block where the block being requested may be present needs to be compared with the TAG field of the MM address.
  - All the possible tags are compared in parallel, as speed is important.
- Mapping Algorithms?
  - Direct mapping requires a single comparison.
  - Associative mapping requires a full associative search over all the TAGs corresponding to all cache blocks.
  - Set associative mapping requires a limited associated search over the TAGs of only the selected set.

- Use of valid bit:
  - There must be a way to know whether a cache block contains valid or garbage information.
  - A valid bit can be added to the TAG, which indicates whether the block contains valid data.
  - If the valid bit is not set, there is no need to match the corresponding TAG.

Q3. Which block should be replaced on a cache miss?

- With fully associative or set associative mapping, there can be several blocks to choose from for replacement when a miss occurs.
- Two primary strategies are used:
  a) **Random**: The candidate block is selected randomly for replacement. This simple strategy tends to spread allocation uniformly.
  b) **Least Recently Used (LRU)**: The block replaced is the one that has not been used for the longest period of time.
     - Makes use of a corollary of temporal locality:
       "If recently used blocks are likely to be used again, then the best candidate for replacement is the least recently used block"

- To implement the LRU algorithm, the cache controller must track the LRU block as the computation proceeds.
- Example: Consider a 4-way set associative cache.
  - For tracking the LRU block within a set, we use a 2-bit counter with every block.
  - When hit occurs:
    - Counter of the referenced block is reset to 0.
    - Counters with values originally lower than the referenced one are incremented by 1, and all others remain unchanged.
  - When miss occurs:
    - If the set is not full, the counter associated with the new block loaded is set to 0, and all other counters are incremented by 1.
    - If the set is full, the block with counter value 3 is removed, the new block put in its place, and the counter set to 0. The other three counters are incremented by 1.

- It may be verified that the counter values of occupied blocks are all distinct.
- An example: